

Namespace ElectricDrill.AstraRpgFramework

Classes

[EntityCore](#)

The core component for any entity in the game. It is the mandatory base class for all entities, providing essential functionality. Provides easy access to the entity's level, stats, and attributes.

Class EntityCore

Namespace: [ElectricDrill.AstraRpgFramework](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

The core component for any entity in the game. It is the mandatory base class for all entities, providing essential functionality. Provides easy access to the entity's level, stats, and attributes.

```
[DisallowMultipleComponent]
public class EntityCore : MonoBehaviour, IEditorValidatable, IInvalidatable, IStatReader,
    IValueContainer<Stat>, IAttributeReader, IValueContainer<Attribute>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← EntityCore

Implements

[IEditorValidatable](#), [IInvalidatable](#), [IStatReader](#), [IValueContainer](#)<[Stat](#)>, [IAttributeReader](#),
[IValueContainer](#)<[Attribute](#)>

Properties

Attributes

Gets the [EntityAttributes](#) component which manages the entity's attributes.

```
public virtual EntityAttributes Attributes { get; }
```

Property Value

[EntityAttributes](#)

Level

Gets the [EntityLevel](#) class which manages the entity's level and experience.

```
public virtual EntityLevel Level { get; }
```

Property Value

[EntityLevel](#)

Name

Gets the name of the entity's GameObject.

```
public string Name { get; }
```

Property Value

string

Stats

Gets the [EntityStats](#) component which manages the entity's stats.

```
public virtual EntityStats Stats { get; }
```

Property Value

[EntityStats](#)

Methods

Awake()

```
protected virtual void Awake()
```

Invalidate()

Marks this object as invalid and raises [OnInvalidated](#).

```
public void Invalidate()
```

Start()

```
protected virtual void Start()
```

Update()

```
protected virtual void Update()
```

Events

OnInvalidated

Raised when this object is invalidated.

```
public event Action OnInvalidated
```

Event Type

Action

Namespace ElectricDrill.AstraRpgFramework.

Attributes

Classes

[Attribute](#)

Represents a character attribute that extends BoundedValue functionality. An attribute is a measurable characteristic such as strength, dexterity, luck, and so on. Can have a min and a max value.

[AttributeSet](#)

A ScriptableObject that defines a collection of attributes for use in character systems. Attribute sets are usually used to group attributes that can be shared across different characters or classes. For example, any character of any class could have the same attribute set, including attributes like strength, dexterity, constitution, intelligence, and luck.

[AttributeSetInstance](#)

Represents a runtime instance of an AttributeSet with actual values for each attribute. Provides methods to manipulate and access attribute values during gameplay.

[AttributeSetInstanceExtensions](#)

Extension methods for converting SerializableDictionary to AttributeSetInstance.

[EntityAttributes](#)

Manages the attributes of an entity. This component calculates attribute values by combining base values, flat modifiers, percentage modifiers, and spent attribute points. It requires an ElectricDrill.AstraRpgFramework.Attributes.EntityAttributes.EntityCore component on the same GameObject.

Structs

[AttributeChangeInfo](#)

Immutable structure that contains information about an attribute value change. Used to track and communicate attribute modifications across the system.

Interfaces

[IAttributeReader](#)

Read-only interface for querying attribute values from an entity. Use [TryGet\(Attribute, out long\)](#) as the safe access path; `Get` requires [Contains\(T\)](#) to be checked first.

[IHasAttributeSet](#)

Interface for objects that expose an AttributeSet definition.

Class Attribute

Namespace: [ElectricDrill.AstraRpgFramework.Attributes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents a character attribute that extends BoundedValue functionality. An attribute is a measurable characteristic such as strength, dexterity, luck, and so on. Can have a min and a max value.

```
[Serializable]
```

```
public class Attribute : BoundedValue
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [BoundedValue](#) ← Attribute

Inherited Members

[BoundedValue.HasMaxValue](#) , [BoundedValue.MaxValue](#) , [BoundedValue.HasMinValue](#) ,
[BoundedValue.MinValue](#)

Struct AttributeChangeInfo

Namespace: [ElectricDrill.AstraRpgFramework.Attributes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Immutable structure that contains information about an attribute value change. Used to track and communicate attribute modifications across the system.

```
public readonly struct AttributeChangeInfo : IHasValueChange<long>
```

Implements

[IHasValueChange](#)<long>

Constructors

AttributeChangeInfo(EntityAttributes, Attribute, long, long)

Initializes a new instance of the AttributeChangeInfo structure.

```
public AttributeChangeInfo(EntityAttributes entityAttributes, Attribute attribute, long  
previousValue, long newValue)
```

Parameters

entityAttributes [EntityAttributes](#)

The EntityAttributes instance that owns the changed attribute

attribute [Attribute](#)

The attribute that was changed

previousValue long

previous value before the change

newValue long

The new value after the change

Properties

AbsAmount

The absolute amount of change between PreviousValue and NewValue.

```
public long AbsAmount { get; }
```

Property Value

long

Attribute

Gets the attribute that was changed.

```
public Attribute Attribute { get; }
```

Property Value

[Attribute](#)

EntityAttributes

Gets the EntityAttributes instance that owns the changed attribute.

```
public EntityAttributes EntityAttributes { get; }
```

Property Value

[EntityAttributes](#)

NewValue

Gets the new value of the attribute after the change.

```
public long NewValue { get; }
```

Property Value

long

PreviousValue

Gets the previous value of the attribute before the change.

```
public long PreviousValue { get; }
```

Property Value

long

Class AttributeSet

Namespace: [ElectricDrill.AstraRpgFramework.Attributes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A ScriptableObject that defines a collection of attributes for use in character systems. Attribute sets are usually used to group attributes that can be shared across different characters or classes. For example, any character of any class could have the same attribute set, including attributes like strength, dexterity, constitution, intelligence, and luck.

```
public class AttributeSet : ScriptableObject, IValueContainer<Attribute>, IEditorValidatable
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← AttributeSet

Implements

[IValueContainer](#)<[Attribute](#)>, [IEditorValidatable](#)

Properties

Attributes

Gets a read-only list of all attributes contained in this set.

```
public IReadOnlyList<Attribute> Attributes { get; }
```

Property Value

IReadOnlyList<[Attribute](#)>

Methods

Contains(Attribute)

Checks whether the specified attribute is contained in this set.

```
public bool Contains(Attribute attribute)
```

Parameters

attribute [Attribute](#)

The attribute to check for

Returns

bool

True if the attribute is in the set, false otherwise

Class AttributeSetInstance

Namespace: [ElectricDrill.AstraRpgFramework.Attributes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents a runtime instance of an AttributeSet with actual values for each attribute. Provides methods to manipulate and access attribute values during gameplay.

```
public class AttributeSetInstance : IValueContainer<Attribute>
```

Inheritance

object ← AttributeSetInstance

Implements

[IValueContainer](#) <[Attribute](#)>

Constructors

AttributeSetInstance(AttributeSet)

Initializes a new instance of AttributeSetInstance based on an AttributeSet. All attributes are initialized with a value of 0.

```
public AttributeSetInstance(AttributeSet attrSet)
```

Parameters

attrSet [AttributeSet](#)

The AttributeSet to base this instance on

Properties

Attributes

Gets the internal dictionary containing all attribute-value pairs.


```
public Dictionary<Attribute, long> Attributes { get; }
```

Property Value

Dictionary<[Attribute](#), long>

this[Attribute]

Gets or sets the value of an attribute using indexer syntax.

```
public long this[Attribute attribute] { get; set; }
```

Parameters

attribute [Attribute](#)

The attribute to access

Property Value

long

The current value when getting, or sets the value when setting

Methods

AddValue(Attribute, long)

Adds a value to the specified attribute. If the attribute doesn't exist, it will be created.

```
public void AddValue(Attribute attribute, long value)
```

Parameters

attribute [Attribute](#)

The attribute to modify

value long

The value to add. Can be negative to decrease the value.

Clone()

Creates a deep copy of this AttributeSetInstance.

```
public AttributeSetInstance Clone()
```

Returns

[AttributeSetInstance](#)

A new AttributeSetInstance with the same attribute values

Contains(Attribute)

Checks whether the specified attribute is contained in this instance.

```
public bool Contains(Attribute stat)
```

Parameters

stat [Attribute](#)

The attribute to check for

Returns

bool

True if the attribute exists, false otherwise

Get(Attribute)

Gets the current value of the specified attribute.

```
public long Get(Attribute attribute)
```

Parameters

attribute [Attribute](#)

The attribute to retrieve

Returns

long

The current value of the attribute

GetAsPercentage(Attribute)

Gets the value of an attribute as a percentage.

```
public Percentage GetAsPercentage(Attribute stat)
```

Parameters

stat [Attribute](#)

The attribute to convert to percentage

Returns

[Percentage](#)

A Percentage representation of the attribute value

GetEnumerator()

Returns an enumerator that iterates through the attribute-value pairs.

```
public IEnumerable<KeyValuePair<Attribute, long>> GetEnumerator()
```

Returns

IEnumerator<KeyValuePair<[Attribute](#), long>>

An enumerator for the attribute-value pairs

Operators

operator +(AttributeSetInstance, AttributeSetInstance)

Adds two AttributeSetInstance objects together, combining their attribute values.

```
public static AttributeSetInstance operator +(AttributeSetInstance a,  
AttributeSetInstance b)
```

Parameters

a [AttributeSetInstance](#)

The first AttributeSetInstance

b [AttributeSetInstance](#)

The second AttributeSetInstance

Returns

[AttributeSetInstance](#)

A new AttributeSetInstance with combined values

explicit operator

AttributeSetInstance(SerializableDictionary<Attribute, long>)

Explicitly converts a SerializableDictionary to an AttributeSetInstance.

```
public static explicit operator AttributeSetInstance(SerializableDictionary<Attribute,  
long> dictionary)
```

Parameters

dictionary [SerializableDictionary](#)<[Attribute](#), long>

The dictionary to convert

Returns

[AttributeSetInstance](#)

A new AttributeSetInstance based on the dictionary

Class AttributeSetInstanceExtensions

Namespace: [ElectricDrill.AstraRpgFramework.Attributes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Extension methods for converting SerializableDictionary to AttributeSetInstance.

```
public static class AttributeSetInstanceExtensions
```

Inheritance

object ← AttributeSetInstanceExtensions

Methods

ToAttributeSetInstance(SerializableDictionary<Attribute, long>, AttributeSet)

Converts a SerializableDictionary of attributes to an AttributeSetInstance. Validates that all attributes from the AttributeSet are present in the dictionary.

```
public static AttributeSetInstance ToAttributeSetInstance(this  
    SerializableDictionary<Attribute, long> dictionary, AttributeSet attributeSet)
```

Parameters

dictionary [SerializableDictionary](#)<[Attribute](#), long>

The dictionary to convert

attributeSet [AttributeSet](#)

The AttributeSet to validate against

Returns

[AttributeSetInstance](#)

A new AttributeSetInstance with values from the dictionary

Class EntityAttributes

Namespace: [ElectricDrill.AstraRpgFramework.Attributes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Manages the attributes of an entity. This component calculates attribute values by combining base values, flat modifiers, percentage modifiers, and spent attribute points. It requires an [ElectricDrill.AstraRpgFramework.Attributes.EntityAttributes.EntityCore](#) component on the same `GameObject`.

```
[DisallowMultipleComponent]
[RequireComponent(typeof(EntityCore))]
public class EntityAttributes : MonoBehaviour, IHasAttributeSet, IValueProvider<Attribute, long>, IAttributeReader, IValueContainer<Attribute>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← EntityAttributes

Implements

[IHasAttributeSet](#), [IValueProvider](#)<[Attribute](#), long>, [IAttributeReader](#), [IValueContainer](#)<[Attribute](#)>

Properties

AttributeSet

Gets the [AttributeSet](#) used by this entity. The source of the attribute set depends on whether [UseBaseAttributesFromClass](#) is true. If true, it returns the attribute set from the [EntityClass](#). Otherwise, it returns the `ElectricDrill.AstraRpgFramework.Attributes.EntityAttributes._fixedBaseAttributeSet`.

```
public virtual AttributeSet AttributeSet { get; }
```

Property Value

[AttributeSet](#)

AvailableAttributePoints

Gets the number of attribute points currently available to spend.

```
public int AvailableAttributePoints { get; }
```

Property Value

int

EntityClass

Gets the class for this entity. The class is used to determine base attributes when ElectricDrill.AstraRpg Framework.Attributes.EntityAttributes._useBaseAttributesFromClass is true.

```
public IClassSource EntityClass { get; }
```

Property Value

[IClassSource](#)

FlatModifiers

Gets the instance of flat modifiers for the attributes. Flat modifiers are added directly to the base attribute values.

```
protected virtual AttributeSetInstance FlatModifiers { get; }
```

Property Value

[AttributeSetInstance](#)

OnAttributeChanged

```
public AttributeChangedGameEvent OnAttributeChanged { get; }
```

Property Value

[AttributeChangedGameEvent](#)

PercentageModifiers

Gets the instance of percentage modifiers for the attributes. Percentage modifiers are applied after flat modifiers.

```
protected virtual AttributeSetInstance PercentageModifiers { get; }
```

Property Value

[AttributeSetInstance](#)

TotalAttributePoints

Gets the total number of attribute points, including both available and spent points.

```
public int TotalAttributePoints { get; }
```

Property Value

int

UseBaseAttributesFromClass

Gets or sets whether to use base attributes from the entity's class. If true, base attributes are derived from the [EntityClass](#). If false, fixed base attributes defined in this component are used.

```
public bool UseBaseAttributesFromClass { get; set; }
```

Property Value

bool

Methods

AddFlatModifier(Attribute, long)

Adds a flat modifier to an attribute.

```
public void AddFlatModifier(Attribute attribute, long value)
```

Parameters

attribute [Attribute](#)

The attribute to modify.

value long

The flat value to add.

AddPercentageModifier(Attribute, Percentage)

Adds a percentage modifier to an attribute.

```
public void AddPercentageModifier(Attribute attribute, Percentage value)
```

Parameters

attribute [Attribute](#)

The attribute to modify.

value [Percentage](#)

The percentage value to add.

Contains(Attribute)

Returns whether **attribute** is tracked by this component's attribute set.

```
public bool Contains(Attribute attribute)
```

Parameters

attribute [Attribute](#)

Returns

bool

Get(Attribute)

Gets the final calculated value of a specific attribute. The final value includes base value, flat and percentage modifiers, and spent attribute points. The result is clamped to the attribute's min/max values.

```
public long Get(Attribute attribute)
```

Parameters

attribute [Attribute](#)

The attribute to retrieve.

Returns

long

The final value of the attribute if the component is enabled (clamped to min/max values); otherwise, 0

GetAllSpentPoints()

Gets a dictionary containing the number of attribute points spent on each attribute.

```
public Dictionary<Attribute, int> GetAllSpentPoints()
```

Returns

Dictionary<[Attribute](#), int>

A dictionary where keys are attributes and values are the points spent on each attribute.

GetBase(Attribute)

Gets the base value of an attribute at the entity's current level. Base attributes don't consider flat or percentage modifiers, nor spent attribute points.

```
public long GetBase(Attribute attribute)
```

Parameters

attribute [Attribute](#)

The attribute to retrieve the base value for.

Returns

long

The base value of the attribute (clamped to its min/max values) if enabled; otherwise, 0

GetSpentOn(Attribute)

Gets the number of attribute points spent on a specific attribute.

```
public int GetSpentOn(Attribute attribute)
```

Parameters

attribute [Attribute](#)

The attribute to check.

Returns

int

The number of points spent on the attribute.

RefundAllSpentPoints()

Refunds all spent attribute points, making them available to spend again.

```
public void RefundAllSpentPoints()
```

RefundFrom(Attribute, int)

Refunds a certain amount of attribute points from a specific attribute.

```
public void RefundFrom(Attribute attribute, int amount)
```

Parameters

attribute [Attribute](#)

The attribute to refund from.

amount int

The number of points to refund.

RefundPointsForAttribute(Attribute)

Refunds all spent attribute points for a specific attribute, making them available to spend again.

```
public void RefundPointsForAttribute(Attribute attribute)
```

Parameters

attribute [Attribute](#)

The attribute to refund points for.

SetFixed(Attribute, long)

Sets a fixed base value for a specific attribute. This is only applicable when [UseBaseAttributesFromClass](#) is false.

```
public void SetFixed(Attribute attribute, long value)
```

Parameters

attribute [Attribute](#)

The attribute to set.

value long

The fixed base value to assign.

SpendOn(Attribute, int)

Spends a certain amount of attribute points on a specific attribute.

```
public void SpendOn(Attribute attribute, int amount)
```

Parameters

attribute [Attribute](#)

The attribute to increase.

amount int

The number of points to spend.

TryGet(Attribute, out long)

Tries to get the final value of an attribute. Returns true if the attribute is contained in the attribute set; otherwise, false.

```
public bool TryGet(Attribute attribute, out long value)
```

Parameters

attribute [Attribute](#)

The attribute to get.

value long

When this method returns, contains the final value of the attribute if the attribute is contained in the attribute set; otherwise, 0.

Returns

bool

true if the attribute is contained in the attribute set; otherwise, false.

Interface IAttributeReader

Namespace: [ElectricDrill.AstraRpgFramework.Attributes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Read-only interface for querying attribute values from an entity. Use [TryGet\(Attribute, out long\)](#) as the safe access path; [Get](#) requires [Contains\(T\)](#) to be checked first.

```
public interface IAttributeReader : IValueContainer<Attribute>
```

Inherited Members

[IValueContainer<Attribute>.Contains\(Attribute\)](#)

Methods

TryGet(Attribute, out long)

Tries to get the final value of an attribute.

```
bool TryGet(Attribute attribute, out long value)
```

Parameters

attribute [Attribute](#)

The attribute to query.

value long

The final value if the attribute is present; otherwise, 0.

Returns

bool

true if the attribute is present; otherwise, **false**.

Interface IHasAttributeSet

Namespace: [ElectricDrill.AstraRpgFramework.Attributes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that expose an AttributeSet definition.

```
public interface IHasAttributeSet
```

Properties

AttributeSet

Gets the attribute set that defines the available attributes.

```
AttributeSet AttributeSet { get; }
```

Property Value

[AttributeSet](#)

Namespace ElectricDrill.AstraRpgFramework.

Classes

Classes

[Class](#)

Represents a character class in the RPG system. It can be used to define how attributes and stats grow over levels.

[EntityClass](#)

Component that represents the class of an entity in the game. Implements [IClassSource](#) to provide access to the entity's class definition.

Interfaces

[IClassSource](#)

Interface for objects that provide a source of a class. In RPG games, a class typically defines the role or profession of an entity, such as a warrior, mage, or rogue.

Class Class

Namespace: [ElectricDrill.AstraRpgFramework.Classes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents a character class in the RPG system. It can be used to define how attributes and stats grow over levels.

```
public class Class : ScriptableObject, IHasStatSet, IHasAttributeSet, IEditorValidatable
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← Class

Implements

[IHasStatSet](#), [IHasAttributeSet](#), [IEditorValidatable](#)

Fields

_attributeSet

The set of attributes associated with this class.

```
[SerializeField]  
protected AttributeSet _attributeSet
```

Field Value

[AttributeSet](#)

_maxHpGrowthFormula

The growth formula for the maximum HP. This is a baseline value that can be used to calculate the maximum HP of characters of this class at different levels.

```
[SerializeField]  
protected GrowthFormula _maxHpGrowthFormula
```

Field Value

[GrowthFormula](#)

_statSet

The set of stats associated with this class.

```
[SerializeField]  
protected StatSet _statSet
```

Field Value

[StatSet](#)

Properties

AttributeSet

Gets the AttributeSet for this class.

```
public virtual AttributeSet AttributeSet { get; }
```

Property Value

[AttributeSet](#)

StatSet

Gets the StatSet for this class.

```
public virtual StatSet StatSet { get; }
```

Property Value

[StatSet](#)

Methods

GetAttributeAt(Attribute, int)

Calculates the value of a specific attribute at a given level.

```
public virtual long GetAttributeAt(Attribute attribute, int level)
```

Parameters

attribute [Attribute](#)

The attribute to calculate.

level int

The level to calculate the attribute value for.

Returns

long

The value of the attribute at the specified level.

GetMaxHpAt(int)

Calculates the maximum HP for a given level.

```
public long GetMaxHpAt(int level)
```

Parameters

level int

The level to calculate the max HP for.

Returns

long

The maximum HP at the specified level.

GetStatAt(Stat, int)

Calculates the value of a specific stat at a given level.

```
public virtual long GetStatAt(Stat stat, int level)
```

Parameters

stat [Stat](#)

The stat to calculate.

level int

The level to calculate the stat value for.

Returns

long

The value of the stat at the specified level.

Exceptions

ArgumentException

Thrown when the growth formula for the stat is not set.

Class EntityClass

Namespace: [ElectricDrill.AstraRpgFramework.Classes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Component that represents the class of an entity in the game. Implements [IClassSource](#) to provide access to the entity's class definition.

```
[DisallowMultipleComponent]  
public class EntityClass : MonoBehaviour, IClassSource
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← EntityClass

Implements

[IClassSource](#)

Properties

Class

Gets or sets the class definition for this entity.

```
public Class Class { get; }
```

Property Value

[Class](#)

Operators

implicit operator Class(EntityClass)

Implicit conversion operator that allows an EntityClass to be used directly as a Class.

```
public static implicit operator Class(EntityClass entityClass)
```

Parameters

entityClass [EntityClass](#)

The EntityClass to convert.

Returns

[Class](#)

The Class associated with the EntityClass.

Interface IClassSource

Namespace: [ElectricDrill.AstraRpgFramework.Classes](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that provide a source of a class. In RPG games, a class typically defines the role or profession of an entity, such as a warrior, mage, or rogue.

```
public interface IClassSource
```

Properties

Class

Gets the class.

```
Class Class { get; }
```

Property Value

[Class](#)

Namespace ElectricDrill.AstraRpgFramework.

Conditions

Classes

[AllConditions](#)

Composite condition that evaluates to **true** only when ALL child conditions are satisfied (logical AND). An empty condition list evaluates to **true**.

[AlwaysFalseCondition](#)

Condition that always evaluates to **false**. Useful to explicitly disable a condition slot during development or as a short-circuit placeholder.

[AlwaysTrueCondition](#)

Condition that always evaluates to **true**. Useful as a default or placeholder in condition chains.

[AnyCondition](#)

Composite condition that evaluates to **true** when ANY child condition is satisfied (logical OR). An empty condition list evaluates to **false**.

[AtLeastNCondition](#)

Composite condition that evaluates to **true** when at least `ElectricDrill.AstraRpgFramework.Conditions.AtLeastNCondition._n` child conditions are satisfied.

With `N = 1` this is equivalent to [AnyCondition](#). With `N = 0` it always evaluates to **true**.

[AtMostNCondition](#)

Composite condition that evaluates to **true** when at most `ElectricDrill.AstraRpgFramework.Conditions.AtMostNCondition._n` child conditions are satisfied.

With `N = 0` this is equivalent to [NoneCondition](#). An empty condition list always evaluates to **true**.

[Condition](#)

Abstract base class for all conditions in the Astra RPG condition system. Conditions are pure predicates evaluated against an [EvaluationContext](#). Subclass and implement [Evaluate\(EvaluationContext\)](#) to define custom logic.

Concrete subclasses must be decorated with `[Serializable]` to participate in Unity's `[SerializeReference]` polymorphic serialization.

[ExactlyNCondition](#)

Composite condition that evaluates to **true** when exactly `ElectricDrill.AstraRpgFramework.Conditions.ExactlyNCondition._n` child conditions are satisfied.

With `N = 1` this is equivalent to an exclusive-OR (XOR) across all children. An empty condition list with `N = 0` evaluates to `true`.

[HasActiveModifierTagCondition](#)

Condition that evaluates to `true` when at least one active modifier on [Holder](#) carries all tags in `ElectricDrill.AstraRpgFramework.Conditions.HasActiveModifierTagCondition._tags`.

Requires an [IActiveModifierTagProvider](#) component on the holder's `GameObject` (typically provided by the Modifiers package). Returns `false` when the holder is `null` or lacks the component.

[HasTagCondition](#)

Condition that checks whether the target entity has the required tags.

The entity is resolved via [ITaggable](#) obtained from a `GetComponent` call on the [ConditionTarget](#) entity. Returns `false` if the target is `null` or does not implement [ITaggable](#).

[HolderLevelCondition](#)

Condition that checks the holder entity's current level against a fixed threshold.

[IsPerformerCondition](#)

Condition that evaluates to `true` when [Performer](#) is the same entity as [Holder](#) (i.e., the effect was self-applied). Returns `false` when [Performer](#) is `null`.

[NoneCondition](#)

Composite condition that evaluates to `true` when NO child condition is satisfied (logical NOR). An empty condition list evaluates to `true`.

[NotCondition](#)

Decorator condition that negates the result of its inner condition (logical NOT). Evaluates to `true` when the inner condition is `false`, and vice-versa. When [Inner](#) is `null`, evaluates to `true`.

[RandomChanceCondition](#)

Condition that evaluates to `true` with the given probability.

`ElectricDrill.AstraRpgFramework.Conditions.RandomChanceCondition._chance` is expressed as a percentage (0–100). A value of 25 means a 25 % chance. Values ≤ 0 always return `false`; values ≥ 100 always return `true`.

[StatThresholdCondition](#)

Condition that checks a stat on the holder entity against a threshold.

When [Absolute](#), the stat's final value is compared directly against `ElectricDrill.AstraRpgFramework.Conditions.StatThresholdCondition._thresholdAbsolute`.

When [Percentage](#), the stat's position within its defined range [`MinValue`, `MaxValue`] is expressed as a

percentage (0–100) and compared against `ElectricDrill.AstraRpgFramework.Conditions.StatThresholdCondition._thresholdPercentage`. Requires the stat to have `HasMaxValue = true`; returns `false` otherwise. When `HasMinValue = false`, 0 is used as the implicit minimum. Returns `false` if the effective range (`MaxValue - MinValue`) is zero to avoid division by zero.

Negative `MinValue` is fully supported: a stat with `MinValue = -100` and `MaxValue = 100` at value 0 evaluates to 50 %.

Structs

[EvaluationContext](#)

Lightweight value type passed to [Evaluate\(EvaluationContext\)](#) on every evaluation. Struct allocation keeps GC pressure at zero even under high tick frequency.

All fields are public for direct assignment via object-initializer syntax. Conditions must treat the context as read-only — mutations are silently discarded (pass-by-value semantics).

Enums

[ComparisonMode](#)

Defines how a numeric value is compared against a threshold.

[ConditionTarget](#)

Specifies which entity in an [EvaluationContext](#) a condition should target.

[TagMatchMode](#)

Controls how a tag set is matched against an entity's tags.

[ThresholdMode](#)

Specifies whether a stat threshold is expressed as a raw absolute value or as a percentage of the stat's maximum value (0–100).

Class AllConditions

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Composite condition that evaluates to **true** only when ALL child conditions are satisfied (logical AND).
An empty condition list evaluates to **true**.

```
[Serializable]  
public class AllConditions : Condition
```

Inheritance

object ← [Condition](#) ← AllConditions

Fields

Conditions

```
[SerializeReference]  
[TypeSelectable]  
public List<Condition> Conditions
```

Field Value

List<[Condition](#)>

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class AlwaysFalseCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Condition that always evaluates to **false**. Useful to explicitly disable a condition slot during development or as a short-circuit placeholder.

```
[Serializable]  
public class AlwaysFalseCondition : Condition
```

Inheritance

object ← [Condition](#) ← AlwaysFalseCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

true if the condition is satisfied; otherwise **false**.

Class AlwaysTrueCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Condition that always evaluates to **true**. Useful as a default or placeholder in condition chains.

```
[Serializable]  
public class AlwaysTrueCondition : Condition
```

Inheritance

object ← [Condition](#) ← AlwaysTrueCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

true if the condition is satisfied; otherwise **false**.

Class AnyCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Composite condition that evaluates to **true** when ANY child condition is satisfied (logical OR). An empty condition list evaluates to **false**.

```
[Serializable]  
public class AnyCondition : Condition
```

Inheritance

object ← [Condition](#) ← AnyCondition

Fields

Conditions

```
[SerializeReference]  
public List<Condition> Conditions
```

Field Value

List<[Condition](#)>

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class AtLeastNCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Composite condition that evaluates to **true** when at least `ElectricDrill.AstraRpgFramework.Conditions.AtLeastNCondition._n` child conditions are satisfied.

With `N = 1` this is equivalent to [AnyCondition](#). With `N = 0` it always evaluates to **true**.

```
[Serializable]  
public class AtLeastNCondition : Condition
```

Inheritance

object ← [Condition](#) ← AtLeastNCondition

Fields

Conditions

```
[SerializeReference]  
public List<Condition> Conditions
```

Field Value

List<[Condition](#)>

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class AtMostNCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Composite condition that evaluates to **true** when at most `ElectricDrill.AstraRpgFramework.Conditions.AtMostNCondition._n` child conditions are satisfied.

With `N = 0` this is equivalent to [NoneCondition](#). An empty condition list always evaluates to **true**.

```
[Serializable]  
public class AtMostNCondition : Condition
```

Inheritance

object ← [Condition](#) ← AtMostNCondition

Fields

Conditions

```
[SerializeReference]  
public List<Condition> Conditions
```

Field Value

List<[Condition](#)>

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Enum ComparisonMode

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Defines how a numeric value is compared against a threshold.

```
public enum ComparisonMode
```

Fields

Above = 3

Strictly greater than the threshold.

AtOrAbove = 2

Greater than or equal to the threshold.

AtOrBelow = 1

Less than or equal to the threshold.

Below = 0

Strictly less than the threshold.

Class Condition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Abstract base class for all conditions in the Astra RPG condition system. Conditions are pure predicates evaluated against an [EvaluationContext](#). Subclass and implement [Evaluate\(EvaluationContext\)](#) to define custom logic.

Concrete subclasses must be decorated with `[Serializable]` to participate in Unity's `[SerializeReference]` polymorphic serialization.

```
[Serializable]  
public abstract class Condition
```

Inheritance

object ← Condition

Derived

[AllConditions](#), [AlwaysFalseCondition](#), [AlwaysTrueCondition](#), [AnyCondition](#), [AtLeastNCondition](#), [AtMostNCondition](#), [ExactlyNCondition](#), [HasActiveModifierTagCondition](#), [HasTagCondition](#), [HolderLevelCondition](#), [IsPerformerCondition](#), [NoneCondition](#), [NotCondition](#), [RandomChanceCondition](#), [StatThresholdCondition](#)

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public abstract bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Enum ConditionTarget

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Specifies which entity in an [EvaluationContext](#) a condition should target.

```
public enum ConditionTarget
```

Fields

```
Holder = 0
```

```
Performer = 1
```

Struct EvaluationContext

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Lightweight value type passed to [Evaluate\(EvaluationContext\)](#) on every evaluation. Struct allocation keeps GC pressure at zero even under high tick frequency.

All fields are public for direct assignment via object-initializer syntax. Conditions must treat the context as read-only — mutations are silently discarded (pass-by-value semantics).

```
public struct EvaluationContext
```

Fields

EventPayload

Optional payload from the triggering event. `null` for tick-based or timer-based evaluation.

```
public object EventPayload
```

Field Value

object

Holder

The entity that holds (owns) the effect (e.g., modifier or ability) being evaluated.

```
public EntityCore Holder
```

Field Value

[EntityCore](#)

Performer

The entity that applied/casted the effect (e.g., modifier or ability). May be `null` or equal to [Holder](#) for self-applied effects.

```
public EntityCore Performer
```

Field Value

[EntityCore](#)

Methods

TryGetPayload<T>(out T)

Attempts to cast [EventPayload](#) to `T`.

```
public bool TryGetPayload<T>(out T payload)
```

Parameters

`payload` `T`

Returns

`bool`

`true` if the payload is a non-null `T`; otherwise `false`.

Type Parameters

`T`

Class ExactlyNCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Composite condition that evaluates to **true** when exactly [ElectricDrill.AstraRpgFramework.Conditions.ExactlyNCondition._n](#) child conditions are satisfied.

With **N = 1** this is equivalent to an exclusive-OR (XOR) across all children. An empty condition list with **N = 0** evaluates to **true**.

```
[Serializable]  
public class ExactlyNCondition : Condition
```

Inheritance

object ← [Condition](#) ← ExactlyNCondition

Fields

Conditions

```
[SerializeReference]  
public List<Condition> Conditions
```

Field Value

List<[Condition](#)>

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

`bool`

`true` if the condition is satisfied; otherwise `false`.

Class HasActiveModifierTagCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Condition that evaluates to **true** when at least one active modifier on [Holder](#) carries all tags in `ElectricDrill.AstraRpgFramework.Conditions.HasActiveModifierTagCondition._tags`.

Requires an [IActiveModifierTagProvider](#) component on the holder's **GameObject** (typically provided by the Modifiers package). Returns **false** when the holder is **null** or lacks the component.

```
[Serializable]  
public class HasActiveModifierTagCondition : Condition
```

Inheritance

object ← [Condition](#) ← HasActiveModifierTagCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

true if the condition is satisfied; otherwise **false**.

Class HasTagCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Condition that checks whether the target entity has the required tags.

The entity is resolved via [ITaggable](#) obtained from a `GetComponent` call on the [ConditionTarget](#) entity. Returns `false` if the target is `null` or does not implement [ITaggable](#).

```
[Serializable]  
public class HasTagCondition : Condition
```

Inheritance

object ← [Condition](#) ← HasTagCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class HolderLevelCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Condition that checks the holder entity's current level against a fixed threshold.

```
[Serializable]  
public class HolderLevelCondition : Condition
```

Inheritance

object ← [Condition](#) ← HolderLevelCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

true if the condition is satisfied; otherwise false.

Class IsPerformerCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Condition that evaluates to **true** when [Performer](#) is the same entity as [Holder](#) (i.e., the effect was self-applied). Returns **false** when [Performer](#) is **null**.

```
[Serializable]  
public class IsPerformerCondition : Condition
```

Inheritance

object ← [Condition](#) ← IsPerformerCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

true if the condition is satisfied; otherwise **false**.

Class NoneCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Composite condition that evaluates to **true** when NO child condition is satisfied (logical NOR). An empty condition list evaluates to **true**.

```
[Serializable]  
public class NoneCondition : Condition
```

Inheritance

object ← [Condition](#) ← NoneCondition

Fields

Conditions

```
[SerializeReference]  
public List<Condition> Conditions
```

Field Value

List<[Condition](#)>

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class NotCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Decorator condition that negates the result of its inner condition (logical NOT). Evaluates to **true** when the inner condition is **false**, and vice-versa. When [Inner](#) is **null**, evaluates to **true**.

```
[Serializable]  
public class NotCondition : Condition
```

Inheritance

object ← [Condition](#) ← NotCondition

Fields

Inner

```
[SerializeReference]  
public Condition Inner
```

Field Value

[Condition](#)

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class RandomChanceCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Condition that evaluates to **true** with the given probability.

ElectricDrill.AstraRpgFramework.Conditions.RandomChanceCondition._chance is expressed as a percentage (0–100). A value of 25 means a 25 % chance. Values ≤ 0 always return **false**; values ≥ 100 always return **true**.

[Serializable]

```
public class RandomChanceCondition : Condition
```

Inheritance

object $\leftarrow$ [Condition](#) $\leftarrow$ RandomChanceCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

ctx [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

true if the condition is satisfied; otherwise **false**.

Class StatThresholdCondition

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Condition that checks a stat on the holder entity against a threshold.

When [Absolute](#), the stat's final value is compared directly against `ElectricDrill.AstraRpgFramework.Conditions.StatThresholdCondition._thresholdAbsolute`.

When [Percentage](#), the stat's position within its defined range [`MinValue`, `MaxValue`] is expressed as a percentage (0–100) and compared against `ElectricDrill.AstraRpgFramework.Conditions.StatThresholdCondition._thresholdPercentage`. Requires the stat to have `HasMaxValue = true`; returns `false` otherwise.

When `HasMinValue = false`, 0 is used as the implicit minimum. Returns `false` if the effective range (`MaxValue - MinValue`) is zero to avoid division by zero.

Negative `MinValue` is fully supported: a stat with `MinValue = -100` and `MaxValue = 100` at value 0 evaluates to 50 %.

`[Serializable]`

```
public class StatThresholdCondition : Condition
```

Inheritance

object ← [Condition](#) ← StatThresholdCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` [EvaluationContext](#)

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Enum TagMatchMode

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Controls how a tag set is matched against an entity's tags.

```
public enum TagMatchMode
```

Fields

AllOf = 1

Every tag in the set must be present.

AnyOf = 0

At least one tag in the set must be present.

Enum ThresholdMode

Namespace: [ElectricDrill.AstraRpgFramework.Conditions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Specifies whether a stat threshold is expressed as a raw absolute value or as a percentage of the stat's maximum value (0–100).

```
public enum ThresholdMode
```

Fields

Absolute = 0

The threshold is a raw stat value.

Percentage = 1

The threshold is a percentage of the stat's maximum value (0–100).

Namespace ElectricDrill.AstraRpgFramework.

Contexts

Interfaces

[IEffectInstigator](#)

Marker interface for anything that can be the specific cause of a game effect: an ability definition, a modifier definition or instance, a passive skill, a game system, etc.

Unlike [IEffectInstigator](#) — which identifies the *entity* responsible for an effect — [IEffectInstigator](#) identifies the specific *thing* that produced it.

Consumers use pattern matching (`instigator is MyType t`) to identify and react to specific instigator types. This design places no constraints on what an instigator must expose, and allows any future type to participate without changes to this interface.

[IHasInstigator](#)

Contract for contexts that carry a reference to the specific cause of the effect.

This complements [IEffectInstigator](#) (which identifies the responsible *entity*) by identifying the specific *thing* — such as an ability, modifier, passive skill, or game system — that produced the effect.

[Instigator](#) is nullable: not every effect has a meaningful specific cause beyond its source entity and category ([IEffectInstigator](#) + DamageSourceSO / HealSourceSO).

Typical usage:

```
// Prevent infinite loops: ignore effects caused by this modifier itself
if (ctx.Instigator == this.Definition) return;
// React only to effects from a specific source type
if (ctx.Instigator is AbilityDefinitionSO ability && ability.IsUltimate)
    ApplyStrengthBuff(ctx.Target);
```

[IHasPerformer](#)

Contract for objects that expose the entity that performed an action (e.g. the damage dealer, the healer, the entity that triggered an effect). May be `null` when the effect has no entity origin (system-initiated, environmental, or engine-driven).

[IHasTarget](#)

[IHasValueChange<T>](#)

[IHasVictim](#)

[Invalidatable](#)

Interface for objects that act as the origin of downstream state and can notify dependents when they are no longer valid — due to entity death, object pooling, aura deactivation, etc.

[OnInvalidated](#) is raised when the object becomes invalid. Subscribers should clean up any state that depends on this object (e.g. modifier instances applied by this source).

[Invalidate\(\)](#) can be called explicitly by any system that governs the object's lifetime (e.g. [EntityHealth](#) calls it on entity death; an object pool calls it before returning an instance to the pool). [OnInvalidated](#) is also raised automatically from [OnDestroy](#) on [EntityCore](#) as a safety net for GameObject destruction.

Interface IEffectInstigator

Namespace: [ElectricDrill.AstraRpgFramework.Contexts](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Marker interface for anything that can be the specific cause of a game effect: an ability definition, a modifier definition or instance, a passive skill, a game system, etc.

Unlike [IEffectInstigator](#) — which identifies the *entity* responsible for an effect — [IEffectInstigator](#) identifies the specific *thing* that produced it.

Consumers use pattern matching (`instigator is MyType t`) to identify and react to specific instigator types. This design places no constraints on what an instigator must expose, and allows any future type to participate without changes to this interface.

```
public interface IEffectInstigator
```

Interface IHasInstigator

Namespace: [ElectricDrill.AstraRpgFramework.Contexts](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Contract for contexts that carry a reference to the specific cause of the effect.

This complements [IEffectInstigator](#) (which identifies the responsible *entity*) by identifying the specific *thing* — such as an ability, modifier, passive skill, or game system — that produced the effect.

[Instigator](#) is nullable: not every effect has a meaningful specific cause beyond its source entity and category ([IEffectInstigator](#) + DamageSourceSO / HealSourceSO).

Typical usage:

```
// Prevent infinite loops: ignore effects caused by this modifier itself
if (ctx.Instigator == this.Definition) return;
// React only to effects from a specific source type
if (ctx.Instigator is AbilityDefinitionSO ability && ability.IsUltimate)
    ApplyStrengthBuff(ctx.Target);
```

```
public interface IHasInstigator
```

Properties

Instigator

The specific thing that caused this effect, or `null` if unspecified.

```
IEffectInstigator Instigator { get; }
```

Property Value

[IEffectInstigator](#)

Interface IHasPerformer

Namespace: [ElectricDrill.AstraRpgFramework.Contexts](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Contract for objects that expose the entity that performed an action (e.g. the damage dealer, the healer, the entity that triggered an effect). May be `null` when the effect has no entity origin (system-initiated, environmental, or engine-driven).

```
public interface IHasPerformer
```

Properties

Performer

```
EntityCore Performer { get; }
```

Property Value

[EntityCore](#)

Interface IHasTarget

Namespace: [ElectricDrill.AstraRpgFramework.Contexts](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
public interface IHasTarget
```

Properties

Target

```
EntityCore Target { get; }
```

Property Value

[EntityCore](#)

Interface IHasValueChange<T>

Namespace: [ElectricDrill.AstraRpgFramework.Contexts](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
public interface IHasValueChange<out T>
```

Type Parameters

T

Properties

AbsAmount

The absolute amount of change between PreviousValue and NewValue.

```
T AbsAmount { get; }
```

Property Value

T

NewValue

```
T NewValue { get; }
```

Property Value

T

PreviousValue

```
T PreviousValue { get; }
```

Property Value

T

Interface IHasVictim

Namespace: [ElectricDrill.AstraRpgFramework.Contexts](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
public interface IHasVictim
```

Properties

Victim

```
EntityCore Victim { get; }
```

Property Value

[EntityCore](#)

Interface IInvalidatable

Namespace: [ElectricDrill.AstraRpgFramework.Contexts](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that act as the origin of downstream state and can notify dependents when they are no longer valid — due to entity death, object pooling, aura deactivation, etc.

[OnInvalidated](#) is raised when the object becomes invalid. Subscribers should clean up any state that depends on this object (e.g. modifier instances applied by this source).

[Invalidate\(\)](#) can be called explicitly by any system that governs the object's lifetime (e.g. [EntityHealth](#) calls it on entity death; an object pool calls it before returning an instance to the pool). [OnInvalidated](#) is also raised automatically from [OnDestroy](#) on [EntityCore](#) as a safety net for GameObject destruction.

```
public interface IInvalidatable
```

Methods

Invalidate()

Marks this object as invalid and raises [OnInvalidated](#).

```
void Invalidate()
```

Events

OnInvalidated

Raised when this object is invalidated.

```
event Action OnInvalidated
```

Event Type

Action

Namespace ElectricDrill.AstraRpgFramework.

Events

Classes

[AttributeChangedGameEvent](#)

Game event that is raised when an attribute changes. Contains information about the attribute that changed, including its previous and new values.

[AttributeChangedGameEventListener](#)

Game event that is raised when an attribute changes. Contains information about the attribute that changed, including its previous and new values.

[EntityCoreGameEvent](#)

Game event that carries EntityCore data. Used for events that need to pass information about an entity's core component.

[EntityCoreGameEventListener](#)

Game event that carries EntityCore data. Used for events that need to pass information about an entity's core component.

[EntityLevelDownGameEvent](#)

Event raised when an entity levels down. Payload: [EntityLevelChangedContext](#).

[EntityLevelDownGameEventListener](#)

Event raised when an entity levels down. Payload: [EntityLevelChangedContext](#).

[EntityLevelUpGameEvent](#)

Event raised when an entity levels up. Payload: [EntityLevelChangedContext](#).

[EntityLevelUpGameEventListener](#)

Event raised when an entity levels up. Payload: [EntityLevelChangedContext](#).

[EntityLeveledDownGameEvent](#)

Event to notify the decrease of the level of an entity. Parameters are:

- the entity that changed level
- the new level

[EntityLeveledDownGameEventListener](#)

Event to notify the decrease of the level of an entity. Parameters are:

- the entity that changed level
- the new level

[EntityLeveledUpGameEvent](#)

Event to notify the increase of the level of an entity. Parameters are:

- the entity that changed level
- the new level

[EntityLeveledUpGameEventListener](#)

Event to notify the increase of the level of an entity. Parameters are:

- the entity that changed level
- the new level

[GameEvent](#)

A ScriptableObject-based event system that allows decoupled communication between game systems. GameEvents can be raised to notify all registered listeners without requiring direct references. Supports both MonoBehaviour-based listeners and code-based System.Action listeners.

[GameEventGeneric1<T>](#)

A generic ScriptableObject-based event system that can carry one parameter of type T. Supports both MonoBehaviour-based listeners and code-based Action listeners.

[GameEventGeneric2<T, U>](#)

A generic ScriptableObject-based event system that can carry two parameters of types T and U. Supports both MonoBehaviour-based listeners and code-based Action listeners.

[GameEventGeneric3<T, U, W>](#)

A generic ScriptableObject-based event system that can carry three parameters of types T, U, and W. Supports both MonoBehaviour-based listeners and code-based Action listeners.

[GameEventGeneric4<T, U, W, K>](#)

A generic ScriptableObject-based event system that can carry four parameters of types T, U, W, and K. Supports both MonoBehaviour-based listeners and code-based Action listeners.

[GameEventListener](#)

MonoBehaviour component that listens to GameEvent objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

[GameEventListenerGeneric1<T>](#)

MonoBehaviour component that listens to GameEventGeneric1 objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

[GameEventListenerGeneric2<T, U>](#)

MonoBehaviour component that listens to GameEventGeneric2 objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

[GameEventListenerGeneric3<T, U, W>](#)

MonoBehaviour component that listens to GameEventGeneric3 objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

[GameEventListenerGeneric4<T, U, W, K>](#)

MonoBehaviour component that listens to GameEventGeneric4 objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

[IntGameEvent](#)

Game event that carries integer data. A general-purpose event for passing integer values between game systems.

[IntGameEventListener](#)

Game event that carries integer data. A general-purpose event for passing integer values between game systems.

[StatChangedGameEvent](#)

Game event that is raised when a stat changes. Contains information about the stat that changed, including its previous and new values.

[StatChangedGameEventListener](#)

Game event that is raised when a stat changes. Contains information about the stat that changed, including its previous and new values.

Interfaces

[IRaisable<T>](#)

Interface for objects that can be raised with one parameter of type T. Defines the contract for events that carry a single piece of data.

[IRaisable<T, U>](#)

Interface for objects that can be raised with two parameters of types T and U. Defines the contract for events that carry two pieces of data.

[IRaisable<T, U, V>](#)

Interface for objects that can be raised with three parameters of types T, U, and V. Defines the contract for events that carry three pieces of data.

[IRaisable<T, U, V, W>](#)

Interface for objects that can be raised with four parameters of types T, U, V, and W. Defines the contract for events that carry four pieces of data.

Class AttributeChangedGameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Game event that is raised when an attribute changes. Contains information about the attribute that changed, including its previous and new values.

```
[CreateAssetMenu(fileName = "AttributeChanged Game Event", menuName = "Astra RPG Framework/Events/Generated/AttributeChanged")]  
public class AttributeChangedGameEvent : GameEventGeneric1<AttributeChangeInfo>, IRaisable<AttributeChangeInfo>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameEventGeneric1<AttributeChangeInfo>](#) ← AttributeChangedGameEvent

Implements

[IRaisable<AttributeChangeInfo>](#)

Inherited Members

[GameEventGeneric1<AttributeChangeInfo>.OnEventRaised](#) ,
[GameEventGeneric1<AttributeChangeInfo>.Raise\(AttributeChangeInfo\)](#) ,
[GameEventGeneric1<AttributeChangeInfo>.RegisterListener\(GameEventListenerGeneric1<AttributeChangeInfo>\)](#) ,
[GameEventGeneric1<AttributeChangeInfo>.UnregisterListener\(GameEventListenerGeneric1<AttributeChangeInfo>\)](#) .

Class AttributeChangedGameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Game event that is raised when an attribute changes. Contains information about the attribute that changed, including its previous and new values.

```
public class AttributeChangedGameEventListener :  
    GameEventListenerGeneric1<AttributeChangeInfo>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ←
[GameEventListenerGeneric1<AttributeChangeInfo>](#) ← AttributeChangedGameEventListener

Inherited Members

[GameEventListenerGeneric1<AttributeChangeInfo>._event](#) ,
[GameEventListenerGeneric1<AttributeChangeInfo>._response](#) ,
[GameEventListenerGeneric1<AttributeChangeInfo>.OnEventRaised\(AttributeChangeInfo\)](#).

Class EntityCoreGameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Game event that carries EntityCore data. Used for events that need to pass information about an entity's core component.

```
[CreateAssetMenu(fileName = "EntityCore Game Event", menuName = "Astra RPG  
Framework/Events/Generated/EntityCore")]  
public class EntityCoreGameEvent : GameEventGeneric1<EntityCore>, IRaisable<EntityCore>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameEventGeneric1<EntityCore>](#) ← EntityCoreGameEvent

Implements

[IRaisable<EntityCore>](#)

Inherited Members

[GameEventGeneric1<EntityCore>.OnEventRaised](#) , [GameEventGeneric1<EntityCore>.Raise\(EntityCore\)](#) ,
[GameEventGeneric1<EntityCore>.RegisterListener\(GameEventListenerGeneric1<EntityCore>\)](#) ,
[GameEventGeneric1<EntityCore>.UnregisterListener\(GameEventListenerGeneric1<EntityCore>\)](#).

Class EntityCoreGameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Game event that carries EntityCore data. Used for events that need to pass information about an entity's core component.

```
public class EntityCoreGameEventListener : GameEventListenerGeneric1<EntityCore>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← [GameEventListenerGeneric1<EntityCore>](#) ← EntityCoreGameEventListener

Inherited Members

[GameEventListenerGeneric1<EntityCore>._event](#) , [GameEventListenerGeneric1<EntityCore>._response](#) , [GameEventListenerGeneric1<EntityCore>.OnEventRaised\(EntityCore\)](#).

Class EntityLevelDownGameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Event raised when an entity levels down. Payload: [EntityLevelChangedContext](#).

```
[CreateAssetMenu(fileName = "EntityLevelDown Game Event", menuName = "Astra RPG Framework/Events/Generated/EntityLevelDown")]  
public class EntityLevelDownGameEvent : GameEventGeneric1<EntityLevelChangedContext>, IRaisable<EntityLevelChangedContext>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameEventGeneric1<EntityLevelChangedContext>](#) ← EntityLevelDownGameEvent

Implements

[IRaisable](#) <[EntityLevelChangedContext](#)>

Inherited Members

[GameEventGeneric1<EntityLevelChangedContext>.OnEventRaised](#) ,
[GameEventGeneric1<EntityLevelChangedContext>.Raise\(EntityLevelChangedContext\)](#) ,
[GameEventGeneric1<EntityLevelChangedContext>.RegisterListener\(GameEventListenerGeneric1<EntityLevelChangedContext>\)](#) ,
[GameEventGeneric1<EntityLevelChangedContext>.UnregisterListener\(GameEventListenerGeneric1<EntityLevelChangedContext>\)](#).

Class EntityLevelDownGameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Event raised when an entity levels down. Payload: [EntityLevelChangedContext](#).

```
public class EntityLevelDownGameEventListener :  
    GameEventListenerGeneric1<EntityLevelChangedContext>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ←
[GameEventListenerGeneric1<EntityLevelChangedContext>](#) ← EntityLevelDownGameEventListener

Inherited Members

[GameEventListenerGeneric1<EntityLevelChangedContext>.
event](#) ,
[GameEventListenerGeneric1<EntityLevelChangedContext>.
response](#) ,
[GameEventListenerGeneric1<EntityLevelChangedContext>.
OnEventRaised\(EntityLevelChangedContext\)](#).

Class EntityLevelUpGameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Event raised when an entity levels up. Payload: [EntityLevelChangedContext](#).

```
[CreateAssetMenu(fileName = "EntityLevelUp Game Event", menuName = "Astra RPG Framework/Events/Generated/EntityLevelUp")]  
public class EntityLevelUpGameEvent : GameEventGeneric1<EntityLevelChangedContext>, IRaisable<EntityLevelChangedContext>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameEventGeneric1<EntityLevelChangedContext>](#) ← EntityLevelUpGameEvent

Implements

[IRaisable](#) <[EntityLevelChangedContext](#)>

Inherited Members

[GameEventGeneric1<EntityLevelChangedContext>.OnEventRaised](#) ,
[GameEventGeneric1<EntityLevelChangedContext>.Raise\(EntityLevelChangedContext\)](#) ,
[GameEventGeneric1<EntityLevelChangedContext>.RegisterListener\(GameEventListenerGeneric1<EntityLevelChangedContext>\)](#) ,
[GameEventGeneric1<EntityLevelChangedContext>.UnregisterListener\(GameEventListenerGeneric1<EntityLevelChangedContext>\)](#).

Class EntityLevelUpGameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Event raised when an entity levels up. Payload: [EntityLevelChangedContext](#).

```
public class EntityLevelUpGameEventListener :  
    GameEventListenerGeneric1<EntityLevelChangedContext>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ←
[GameEventListenerGeneric1<EntityLevelChangedContext>](#) ← EntityLevelUpGameEventListener

Inherited Members

[GameEventListenerGeneric1<EntityLevelChangedContext>.event](#) ,
[GameEventListenerGeneric1<EntityLevelChangedContext>.response](#) ,
[GameEventListenerGeneric1<EntityLevelChangedContext>.OnEventRaised\(EntityLevelChangedContext\)](#)

Class EntityLeveledDownGameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Event to notify the decrease of the level of an entity. Parameters are:

- the entity that changed level
- the new level

```
[Obsolete("Use EntityLevelDownGameEvent instead.")]  
[CreateAssetMenu(fileName = "EntityLeveledDown Game Event", menuName = "Astra RPG  
Framework/Events/Generated/EntityLeveledDown")]  
public class EntityLeveledDownGameEvent : GameEventGeneric2<EntityCore, int>,  
IRaisable<EntityCore, int>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameEventGeneric2](#)<[EntityCore](#), int> ←
EntityLeveledDownGameEvent

Implements

[IRaisable](#)<[EntityCore](#), int>

Inherited Members

[GameEventGeneric2](#)<[EntityCore](#), int>.OnEventRaised ,
[GameEventGeneric2](#)<[EntityCore](#), int>.Raise([EntityCore](#), int) ,
[GameEventGeneric2](#)<[EntityCore](#), int>.RegisterListener([GameEventListenerGeneric2](#)<[EntityCore](#), int>) ,
[GameEventGeneric2](#)<[EntityCore](#), int>.UnregisterListener([GameEventListenerGeneric2](#)<[EntityCore](#), int>).

Class EntityLeveledDownGameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Event to notify the decrease of the level of an entity. Parameters are:

- the entity that changed level
- the new level

```
[Obsolete("Use EntityLevelDownGameEventListener instead.")]
```

```
public class EntityLeveledDownGameEventListener : GameEventListenerGeneric2<EntityCore, int>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ←

[GameEventListenerGeneric2](#)<[EntityCore](#), int> ← EntityLeveledDownGameEventListener

Inherited Members

[GameEventListenerGeneric2](#)<[EntityCore](#), int>. [event](#) ,

[GameEventListenerGeneric2](#)<[EntityCore](#), int>. [response](#) ,

[GameEventListenerGeneric2](#)<[EntityCore](#), int>. [OnEventRaised](#)([EntityCore](#), int).

Class EntityLeveledUpGameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Event to notify the increase of the level of an entity. Parameters are:

- the entity that changed level
- the new level

```
[Obsolete("Use EntityLevelUpGameEvent instead.")]  
[CreateAssetMenu(fileName = "EntityLeveledUp Game Event", menuName = "Astra RPG  
Framework/Events/Generated/EntityLeveledUp")]  
public class EntityLeveledUpGameEvent : GameEventGeneric2<EntityCore, int>,  
IRaisable<EntityCore, int>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameEventGeneric2](#)<[EntityCore](#), int> ←
EntityLeveledUpGameEvent

Implements

[IRaisable](#)<[EntityCore](#), int>

Inherited Members

[GameEventGeneric2](#)<[EntityCore](#), int>.OnEventRaised ,
[GameEventGeneric2](#)<[EntityCore](#), int>.Raise([EntityCore](#), int) ,
[GameEventGeneric2](#)<[EntityCore](#), int>.RegisterListener([GameEventListenerGeneric2](#)<[EntityCore](#), int>) ,
[GameEventGeneric2](#)<[EntityCore](#), int>.UnregisterListener([GameEventListenerGeneric2](#)<[EntityCore](#), int>).

Class EntityLeveledUpGameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Event to notify the increase of the level of an entity. Parameters are:

- the entity that changed level
- the new level

```
[Obsolete("Use EntityLevelUpGameEventListener instead.")]
```

```
public class EntityLeveledUpGameEventListener : GameEventListenerGeneric2<EntityCore, int>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ←
[GameEventListenerGeneric2](#)<[EntityCore](#), int> ← EntityLeveledUpGameEventListener

Inherited Members

[GameEventListenerGeneric2](#)<[EntityCore](#), int>. [event](#) ,
[GameEventListenerGeneric2](#)<[EntityCore](#), int>. [response](#) ,
[GameEventListenerGeneric2](#)<[EntityCore](#), int>. [OnEventRaised](#)([EntityCore](#), int).

Class GameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A ScriptableObject-based event system that allows decoupled communication between game systems. GameEvents can be raised to notify all registered listeners without requiring direct references. Supports both MonoBehaviour-based listeners and code-based System.Action listeners.

```
[CreateAssetMenu(fileName = "New Game Event", menuName = "Astra RPG  
Framework/Events/Game Event")]  
public class GameEvent : ScriptableObject
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← GameEvent

Methods

Raise()

Raises the event, notifying all registered listeners. Listeners are processed in reverse order to handle potential modifications during iteration.

```
public void Raise()
```

RegisterListener(GameEventListener)

Registers a listener to be notified when this event is raised. Duplicate listeners are automatically prevented.

```
public void RegisterListener(GameEventListener listener)
```

Parameters

listener [GameEventListener](#)

The `GameEventListener` to register.

UnregisterListener(`GameEventListener`)

Unregisters a listener so it will no longer be notified when this event is raised.

```
public void UnregisterListener(GameEventListener listener)
```

Parameters

listener [GameEventListener](#)

The `GameEventListener` to unregister.

Events

OnEventRaised

Event that can be subscribed to from code using standard C# event syntax. Automatically handles registration and unregistration of Action listeners.

```
public virtual event Action OnEventRaised
```

Event Type

Action

Class GameEventGeneric1<T>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A generic ScriptableObject-based event system that can carry one parameter of type T. Supports both MonoBehaviour-based listeners and code-based Action listeners.

```
public abstract class GameEventGeneric1<T> : ScriptableObject, IRaisable<T>
```

Type Parameters

T

The type of data this event carries.

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← GameEventGeneric1<T>

Implements

[IRaisable](#)<T>

Derived

[AttributeChangedGameEvent](#), [EntityCoreGameEvent](#), [EntityLevelDownGameEvent](#),
[EntityLevelUpGameEvent](#), [IntGameEvent](#), [StatChangedGameEvent](#)

Methods

Raise(T)

Raises the event with the specified context, notifying all registered listeners. Both MonoBehaviour listeners and Action listeners are notified.

```
public virtual void Raise(T context)
```

Parameters

context T

The data to pass to all listeners.

RegisterListener(GameEventListenerGeneric1<T>)

Registers a MonoBehaviour-based listener to be notified when this event is raised. Duplicate listeners are automatically prevented.

```
public void RegisterListener(GameEventListenerGeneric1<T> listener)
```

Parameters

listener [GameEventListenerGeneric1<T>](#)

The GameEventListenerGeneric1 to register.

UnregisterListener(GameEventListenerGeneric1<T>)

Unregisters a MonoBehaviour-based listener so it will no longer be notified when this event is raised.

```
public void UnregisterListener(GameEventListenerGeneric1<T> listener)
```

Parameters

listener [GameEventListenerGeneric1<T>](#)

The GameEventListenerGeneric1 to unregister.

Events

OnEventRaised

Event that can be subscribed to from code using standard C# event syntax. Automatically handles registration and unregistration of Action listeners.

```
public virtual event Action<T> OnEventRaised
```


Event Type

Action<T>

Class GameEventGeneric2<T, U>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A generic ScriptableObject-based event system that can carry two parameters of types T and U. Supports both MonoBehaviour-based listeners and code-based Action listeners.

```
public abstract class GameEventGeneric2<T, U> : ScriptableObject, IRaisable<T, U>
```

Type Parameters

T

The type of the first parameter this event carries.

U

The type of the second parameter this event carries.

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← GameEventGeneric2<T, U>

Implements

[IRaisable](#)<T, U>

Derived

[EntityLeveledDownGameEvent](#), [EntityLeveledUpGameEvent](#)

Methods

Raise(T, U)

Raises the event with the specified contexts, notifying all registered listeners. Both MonoBehaviour listeners and Action listeners are notified.

```
public virtual void Raise(T context1, U context2)
```

Parameters

context1 T

The first parameter to pass to all listeners.

context2 U

The second parameter to pass to all listeners.

RegisterListener(GameEventListenerGeneric2<T, U>)

Registers a MonoBehaviour-based listener to be notified when this event is raised. Duplicate listeners are automatically prevented.

```
public void RegisterListener(GameEventListenerGeneric2<T, U> listener)
```

Parameters

listener [GameEventListenerGeneric2<T, U>](#)

The GameEventListenerGeneric2 to register.

UnregisterListener(GameEventListenerGeneric2<T, U>)

Unregisters a MonoBehaviour-based listener so it will no longer be notified when this event is raised.

```
public void UnregisterListener(GameEventListenerGeneric2<T, U> listener)
```

Parameters

listener [GameEventListenerGeneric2<T, U>](#)

The GameEventListenerGeneric2 to unregister.

Events

OnEventRaised

Event that can be subscribed to from code using standard C# event syntax. Automatically handles registration and unregistration of Action listeners.

```
public virtual event Action<T, U> OnEventRaised
```

Event Type

Action<T, U>

Class GameEventGeneric3<T, U, W>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A generic ScriptableObject-based event system that can carry three parameters of types T, U, and W. Supports both MonoBehaviour-based listeners and code-based Action listeners.

```
public abstract class GameEventGeneric3<T, U, W> : ScriptableObject, IRaisable<T, U, W>
```

Type Parameters

T

The type of the first parameter this event carries.

U

The type of the second parameter this event carries.

W

The type of the third parameter this event carries.

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← GameEventGeneric3<T, U, W>

Implements

[IRaisable](#)<T, U, W>

Methods

Raise(T, U, W)

Raises the event with the specified contexts, notifying all registered listeners. Both MonoBehaviour listeners and Action listeners are notified.

```
public virtual void Raise(T contextT, U contextU, W contextW)
```

Parameters

contextT T

The first parameter to pass to all listeners.

contextU U

The second parameter to pass to all listeners.

contextW W

The third parameter to pass to all listeners.

RegisterListener(GameEventListenerGeneric3<T, U, W>)

Registers a MonoBehaviour-based listener to be notified when this event is raised. Duplicate listeners are automatically prevented.

```
public void RegisterListener(GameEventListenerGeneric3<T, U, W> listener)
```

Parameters

listener [GameEventListenerGeneric3](#)<T, U, W>

The GameEventListenerGeneric3 to register.

UnregisterListener(GameEventListenerGeneric3<T, U, W>)

Unregisters a MonoBehaviour-based listener so it will no longer be notified when this event is raised.

```
public void UnregisterListener(GameEventListenerGeneric3<T, U, W> listener)
```

Parameters

listener [GameEventListenerGeneric3](#)<T, U, W>

The GameEventListenerGeneric3 to unregister.

Events

OnEventRaised

Event that can be subscribed to from code using standard C# event syntax. Automatically handles registration and unregistration of Action listeners.

```
public virtual event Action<T, U, W> OnEventRaised
```

Event Type

Action<T, U, W>

Class GameEventGeneric4<T, U, W, K>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A generic ScriptableObject-based event system that can carry four parameters of types T, U, W, and K. Supports both MonoBehaviour-based listeners and code-based Action listeners.

```
public abstract class GameEventGeneric4<T, U, W, K> : ScriptableObject, IRaisable<T, U, W, K>
```

Type Parameters

T

The type of the first parameter this event carries.

U

The type of the second parameter this event carries.

W

The type of the third parameter this event carries.

K

The type of the fourth parameter this event carries.

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← GameEventGeneric4<T, U, W, K>

Implements

[IRaisable](#)<T, U, W, K>

Methods

Raise(T, U, W, K)

Raises the event with the specified contexts, notifying all registered listeners. Both MonoBehaviour listeners and Action listeners are notified.


```
public virtual void Raise(T contextT, U contextU, W contextW, K contextK)
```

Parameters

contextT T

The first parameter to pass to all listeners.

contextU U

The second parameter to pass to all listeners.

contextW W

The third parameter to pass to all listeners.

contextK K

The fourth parameter to pass to all listeners.

RegisterListener(GameEventListenerGeneric4<T, U, W, K>)

Registers a MonoBehaviour-based listener to be notified when this event is raised. Duplicate listeners are automatically prevented.

```
public void RegisterListener(GameEventListenerGeneric4<T, U, W, K> listener)
```

Parameters

listener [GameEventListenerGeneric4](#)<T, U, W, K>

The GameEventListenerGeneric4 to register.

UnregisterListener(GameEventListenerGeneric4<T, U, W, K>)

Unregisters a MonoBehaviour-based listener so it will no longer be notified when this event is raised.

```
public void UnregisterListener(GameEventListenerGeneric4<T, U, W, K> listener)
```

Parameters

listener [GameEventListenerGeneric4](#)<T, U, W, K>

The GameEventListenerGeneric4 to unregister.

Events

OnEventRaised

Event that can be subscribed to from code using standard C# event syntax. Automatically handles registration and unregistration of Action listeners.

```
public virtual event Action<T, U, W, K> OnEventRaised
```

Event Type

Action<T, U, W, K>

Class GameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

MonoBehaviour component that listens to GameEvent objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

```
public class GameEventListener : MonoBehaviour
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← GameEventListener

Methods

OnEventRaised()

Called by the GameEvent when it is raised. Invokes the configured UnityEvent response.

```
public void OnEventRaised()
```

Class GameEventListenerGeneric1<T>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

MonoBehaviour component that listens to GameEventGeneric1 objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

```
public class GameEventListenerGeneric1<T> : MonoBehaviour
```

Type Parameters

T

The type of parameter the listened event carries.

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← GameEventListenerGeneric1<T>

Derived

[AttributeChangedGameEventListener](#), [EntityCoreGameEventListener](#), [EntityLevelDownGameEventListener](#), [EntityLevelUpGameEventListener](#), [IntGameEventListener](#), [StatChangedGameEventListener](#)

Fields

_event

[SerializeField]

```
protected GameEventGeneric1<T> _event
```

Field Value

[GameEventGeneric1](#)<T>

_response

```
[SerializeField]  
protected UnityEvent<T> _response
```

Field Value

UnityEvent<T>

Methods

OnEventRaised(T)

Called by the GameEvent when it is raised. Invokes the configured UnityEvent response with the provided context.

```
public void OnEventRaised(T context)
```

Parameters

context T

The data passed from the event.

Class GameEventListenerGeneric2<T, U>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

MonoBehaviour component that listens to GameEventGeneric2 objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

```
public class GameEventListenerGeneric2<T, U> : MonoBehaviour
```

Type Parameters

T

The type of the first parameter the listened event carries.

U

The type of the second parameter the listened event carries.

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ←
GameEventListenerGeneric2<T, U>

Derived

[EntityLeveledDownGameEventListener](#), [EntityLeveledUpGameEventListener](#)

Fields

_event

[SerializeField]

```
protected GameEventGeneric2<T, U> _event
```

Field Value

[GameEventGeneric2](#)<T, U>

`_response`

`[SerializeField]`

`protected UnityEvent<T, U> _response`

Field Value

UnityEvent<T, U>

Methods

OnEventRaised(T, U)

Called by the GameEvent when it is raised. Invokes the configured UnityEvent response with the provided contexts.

```
public void OnEventRaised(T contextT, U contextU)
```

Parameters

`contextT` T

The first parameter passed from the event.

`contextU` U

The second parameter passed from the event.

Class `GameEventListenerGeneric3<T, U, W>`

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

MonoBehaviour component that listens to `GameEventGeneric3` objects and responds with `UnityEvents`. Automatically registers and unregisters itself when enabled/disabled.

```
public class GameEventListenerGeneric3<T, U, W> : MonoBehaviour
```

Type Parameters

T

The type of the first parameter the listened event carries.

U

The type of the second parameter the listened event carries.

W

The type of the third parameter the listened event carries.

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← `GameEventListenerGeneric3<T, U, W>`

Fields

`_event`

```
[SerializeField]  
protected GameEventGeneric3<T, U, W> _event
```

Field Value

[GameEventGeneric3](#)<T, U, W>

_response

[SerializeField]

protected UnityEvent<T, U, W> _response

Field Value

UnityEvent<T, U, W>

Methods

OnEventRaised(T, U, W)

Called by the GameEvent when it is raised. Invokes the configured UnityEvent response with the provided contexts.

```
public void OnEventRaised(T contextT, U contextU, W contextW)
```

Parameters

contextT T

The first parameter passed from the event.

contextU U

The second parameter passed from the event.

contextW W

The third parameter passed from the event.

Class GameEventListenerGeneric4<T, U, W, K>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

MonoBehaviour component that listens to GameEventGeneric4 objects and responds with UnityEvents. Automatically registers and unregisters itself when enabled/disabled.

```
public class GameEventListenerGeneric4<T, U, W, K> : MonoBehaviour
```

Type Parameters

T

The type of the first parameter the listened event carries.

U

The type of the second parameter the listened event carries.

W

The type of the third parameter the listened event carries.

K

The type of the fourth parameter the listened event carries.

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ←
GameEventListenerGeneric4<T, U, W, K>

Fields

_event

[SerializeField]

```
protected GameEventGeneric4<T, U, W, K> _event
```

Field Value

[GameEventGeneric4](#)<T, U, W, K>

_response

[SerializeField]

protected UnityEvent<T, U, W, K> _response

Field Value

UnityEvent<T, U, W, K>

Methods

OnEventRaised(T, U, W, K)

Called by the GameEvent when it is raised. Invokes the configured UnityEvent response with the provided contexts.

```
public void OnEventRaised(T contextT, U contextU, W contextW, K contextK)
```

Parameters

contextT T

The first parameter passed from the event.

contextU U

The second parameter passed from the event.

contextW W

The third parameter passed from the event.

contextK K

The fourth parameter passed from the event.

Interface IRaisable<T>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that can be raised with one parameter of type T. Defines the contract for events that carry a single piece of data.

```
public interface IRaisable<T>
```

Type Parameters

T

The type of data this raisable object carries.

Methods

Raise(T)

Raises the event with the specified context.

```
void Raise(T context)
```

Parameters

context T

The data to pass when raising the event.

Interface IRaisable<T, U>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that can be raised with two parameters of types T and U. Defines the contract for events that carry two pieces of data.

```
public interface IRaisable<T, U>
```

Type Parameters

T

The type of the first parameter.

U

The type of the second parameter.

Methods

Raise(T, U)

Raises the event with the specified contexts.

```
void Raise(T context1, U context2)
```

Parameters

context1 T

The first parameter to pass when raising the event.

context2 U

The second parameter to pass when raising the event.

Interface IRaisable<T, U, V>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that can be raised with three parameters of types T, U, and V. Defines the contract for events that carry three pieces of data.

```
public interface IRaisable<T, U, V>
```

Type Parameters

T

The type of the first parameter.

U

The type of the second parameter.

V

The type of the third parameter.

Methods

Raise(T, U, V)

Raises the event with the specified contexts.

```
void Raise(T context1, U context2, V context3)
```

Parameters

context1 T

The first parameter to pass when raising the event.

context2 U

The second parameter to pass when raising the event.

`context3` V

The third parameter to pass when raising the event.

Interface IRaisable<T, U, V, W>

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that can be raised with four parameters of types T, U, V, and W. Defines the contract for events that carry four pieces of data.

```
public interface IRaisable<T, U, V, W>
```

Type Parameters

T

The type of the first parameter.

U

The type of the second parameter.

V

The type of the third parameter.

W

The type of the fourth parameter.

Methods

Raise(T, U, V, W)

Raises the event with the specified contexts.

```
void Raise(T context1, U context2, V context3, W context4)
```

Parameters

context1 T

The first parameter to pass when raising the event.

context2 U

The second parameter to pass when raising the event.

context3 V

The third parameter to pass when raising the event.

context4 W

The fourth parameter to pass when raising the event.

Class IntGameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Game event that carries integer data. A general-purpose event for passing integer values between game systems.

```
[CreateAssetMenu(fileName = "Int Game Event", menuName = "Astra RPG  
Framework/Events/Generated/Int")]  
public class IntGameEvent : GameEventGeneric1<int>, IRaisable<int>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameEventGeneric1](#)<int> ← IntGameEvent

Implements

[IRaisable](#)<int>

Inherited Members

[GameEventGeneric1](#)<int>.OnEventRaised, [GameEventGeneric1](#)<int>.Raise(int),
[GameEventGeneric1](#)<int>.RegisterListener(GameEventListenerGeneric1<int>),
[GameEventGeneric1](#)<int>.UnregisterListener(GameEventListenerGeneric1<int>).

Class IntGameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Game event that carries integer data. A general-purpose event for passing integer values between game systems.

```
public class IntGameEventListener : GameEventListenerGeneric1<int>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← [GameEventListenerGeneric1](#)<int> ← IntGameEventListener

Inherited Members

[GameEventListenerGeneric1](#)<int>._event , [GameEventListenerGeneric1](#)<int>._response , [GameEventListenerGeneric1](#)<int>.OnEventRaised(int)

Class StatChangedGameEvent

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Game event that is raised when a stat changes. Contains information about the stat that changed, including its previous and new values.

```
[CreateAssetMenu(fileName = "StatChanged Game Event", menuName = "Astra RPG  
Framework/Events/Generated/StatChanged")]  
public class StatChangedGameEvent : GameEventGeneric1<StatChangeInfo>,  
IRaisable<StatChangeInfo>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameEventGeneric1<StatChangeInfo>](#) ←
StatChangedGameEvent

Implements

[IRaisable](#) <[StatChangeInfo](#)>

Inherited Members

[GameEventGeneric1<StatChangeInfo>.OnEventRaised](#) ,
[GameEventGeneric1<StatChangeInfo>.Raise\(StatChangeInfo\)](#) ,
[GameEventGeneric1<StatChangeInfo>.RegisterListener\(GameEventListenerGeneric1<StatChangeInfo>\)](#) ,
[GameEventGeneric1<StatChangeInfo>.UnregisterListener\(GameEventListenerGeneric1<StatChangeInfo>\)](#)
).

Class StatChangedGameEventListener

Namespace: [ElectricDrill.AstraRpgFramework.Events](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Game event that is raised when a stat changes. Contains information about the stat that changed, including its previous and new values.

```
public class StatChangedGameEventListener : GameEventListenerGeneric1<StatChangeInfo>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← [GameEventListenerGeneric1<StatChangeInfo>](#) ← StatChangedGameEventListener

Inherited Members

[GameEventListenerGeneric1<StatChangeInfo>._event](#) ,
[GameEventListenerGeneric1<StatChangeInfo>._response](#) ,
[GameEventListenerGeneric1<StatChangeInfo>.OnEventRaised\(StatChangeInfo\)](#).

Namespace ElectricDrill.AstraRpgFramework. Events.Contexts

Classes

[EntityLevelChangedContext](#)

Class EntityLevelChangedContext

Namespace: [ElectricDrill.AstraRpgFramework.Events.Contexts](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
public class EntityLevelChangedContext : IHasTarget, IHasValueChange<int>
```

Inheritance

object ← EntityLevelChangedContext

Implements

[IHasTarget](#), [IHasValueChange](#)<int>

Constructors

EntityLevelChangedContext(EntityCore, int, int)

```
public EntityLevelChangedContext(EntityCore target, int previousValue, int newValue)
```

Parameters

target [EntityCore](#)

previousValue int

newValue int

Properties

AbsAmount

The absolute amount of change between PreviousValue and NewValue.

```
public int AbsAmount { get; }
```

Property Value

int

NewValue

```
public int NewValue { get; }
```

Property Value

int

PreviousValue

```
public int PreviousValue { get; }
```

Property Value

int

Target

```
public EntityCore Target { get; }
```

Property Value

[EntityCore](#)

Namespace ElectricDrill.AstraRpgFramework.

Experience

Classes

[EntityLevel](#)

Represents the level and experience system of an entity in the RPG framework. Handles experience gain, level progression, and experience-based calculations. Implements [ILevelable](#) to provide standard leveling functionality.

[ExpSource](#)

A MonoBehaviour component that provides a source of experience points. Implements [IExpSource](#).

Interfaces

[IExpSource](#)

Interface for objects that can provide experience points. Defines the contract for experience sources that can be harvested.

[ILevelable](#)

Interface for entities that have a leveling system based on experience points. Provides the core functionality for level progression and experience management.

Class EntityLevel

Namespace: [ElectricDrill.AstraRpgFramework.Experience](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents the level and experience system of an entity in the RPG framework. Handles experience gain, level progression, and experience-based calculations. Implements [ILevelable](#) to provide standard leveling functionality.

```
[Serializable]  
public class EntityLevel : ILevelable
```

Inheritance

object ← EntityLevel

Implements

[ILevelable](#)

Properties

CurrentTotalExperience

Gets the current total experience of the entity. This represents all experience gained since the entity was created.

```
public long CurrentTotalExperience { get; }
```

Property Value

long

Level

Gets or sets the current level of the entity. Level must be between 1 and the maximum level. Setting the level will trigger the OnLevelUp or OnLevelDown event for each level changed.

```
public virtual int Level { get; set; }
```

Property Value

int

MaxLevel

Gets the maximum level this entity can reach.

```
public int MaxLevel { get; }
```

Property Value

int

Methods

AddExp(long)

Adds experience to the entity. The experience will be modified by any experience gain modifiers. If the entity gains enough experience to level up, the OnLevelUp event will be raised. Can trigger multiple level ups if enough experience is added.

```
public void AddExp(long amount)
```

Parameters

amount long

The base amount of experience to add before modifiers are applied.

CurrentLevelTotalExperience()

Gets the total experience required to reach the current level. For level 1, this returns 0. For higher levels, it uses the experience growth formula.

```
public long CurrentLevelTotalExperience()
```

Returns

long

The total experience required for the current level.

NextLevelTotalExperience()

Gets the total experience required to reach the next level. If the entity is already at the maximum level, returns the experience for the current level.

```
public long NextLevelTotalExperience()
```

Returns

long

The total experience required for the next level, or current level experience if at max level.

RemoveExp(long)

Removes experience from the entity. The experience is removed as a flat amount without modifiers. If the entity loses enough experience to level down, the OnLevelDown event will be raised. Can trigger multiple level downs if enough experience is removed. Experience cannot go below 0 and level cannot go below 1.

```
public void RemoveExp(long amount)
```

Parameters

amount long

The amount of experience to remove.

ResetToLevelOne()

Resets the entity to level 1 with 0 experience. This will trigger OnLevelDown events for each level lost.

```
public void ResetToLevelOne()
```

SetTotalCurrentExp(long)

Sets the total current experience and automatically updates the level to match. The level will be calculated based on the experience growth formula. This method properly triggers OnLevelUp or OnLevelDown events to ensure all dependent components (like EntityAttributes) react correctly to the level change.

```
public void SetTotalCurrentExp(long totalCurrentExperience)
```

Parameters

totalCurrentExperience long

The total experience to set. Will be clamped between 0 and max level experience.

ValidateExperience()

Validates that the current total experience corresponds to the current level. If the experience doesn't match the level, it will be reset to the base experience for the current level. This method is useful for debugging and ensuring data consistency.

```
public void ValidateExperience()
```

Events

OnEntityLevelDown

Event raised when the entity levels down.

```
public virtual event Action<EntityLevelChangedContext> OnEntityLevelDown
```

Event Type

Action<[EntityLevelChangedContext](#)>

OnEntityLevelUp

Event raised when the entity levels up.

```
public virtual event Action<EntityLevelChangedContext> OnEntityLevelUp
```

Event Type

Action<[EntityLevelChangedContext](#)>

OnLevelDown

Event raised when the entity levels down. The parameters are the EntityCore that leveled down and its new level respectively.

```
[Obsolete("Use OnEntityLevelDown instead.")]  
public virtual event Action<EntityCore, int> OnLevelDown
```

Event Type

Action<[EntityCore](#), int>

OnLevelUp

Event raised when the entity levels up. The parameters are the EntityCore that leveled up and its new level respectively.

```
[Obsolete("Use OnEntityLevelUp instead.")]  
public virtual event Action<EntityCore, int> OnLevelUp
```

Event Type

Action<[EntityCore](#), int>

Operators

implicit operator int(EntityLevel)

Implicitly converts an EntityLevel to its integer level value. Allows EntityLevel objects to be used directly as integers in calculations.

```
public static implicit operator int(EntityLevel entityLevel)
```

Parameters

entityLevel [EntityLevel](#)

The EntityLevel instance to convert.

Returns

int

The current level as an integer.

Class ExpSource

Namespace: [ElectricDrill.AstraRpgFramework.Experience](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A MonoBehaviour component that provides a source of experience points. Implements [IExpSource](#).

```
public class ExpSource : MonoBehaviour, IExpSource
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← ExpSource

Implements

[IExpSource](#)

Properties

Exp

Gets the experience points from this source.

```
public long Exp { get; }
```

Property Value

long

Harvested

Gets or sets whether this experience source has been harvested.

```
public bool Harvested { get; set; }
```

Property Value

bool

Interface IExpSource

Namespace: [ElectricDrill.AstraRpgFramework.Experience](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that can provide experience points. Defines the contract for experience sources that can be harvested.

```
public interface IExpSource
```

Properties

Exp

Gets the experience points available from this source. The behavior may vary based on implementation and harvest state.

```
long Exp { get; }
```

Property Value

long

Harvested

Gets or sets whether this experience source has been harvested. Used to track if the experience has already been collected.

```
bool Harvested { get; set; }
```

Property Value

bool

Interface ILevelable

Namespace: [ElectricDrill.AstraRpgFramework.Experience](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for entities that have a leveling system based on experience points. Provides the core functionality for level progression and experience management.

```
public interface ILevelable
```

Properties

CurrentTotalExperience

Gets the current total experience of the entity. This represents all experience gained since creation.

```
long CurrentTotalExperience { get; }
```

Property Value

long

Level

Gets or sets the current level of the entity. Setting the level may trigger level-up events and related effects.

```
int Level { get; set; }
```

Property Value

int

Methods

AddExp(long)

Adds experience points to the entity. May cause the entity to level up if enough experience is gained.

```
void AddExp(long amount)
```

Parameters

amount long

The amount of experience to add.

CurrentLevelTotalExperience()

Gets the total experience required to reach the current level.

```
long CurrentLevelTotalExperience()
```

Returns

long

The experience threshold for the current level.

NextLevelTotalExperience()

Gets the total experience required to reach the next level.

```
long NextLevelTotalExperience()
```

Returns

long

The experience threshold for the next level.

RemoveExp(long)

Removes experience points from the entity without applying modifiers. May cause the entity to level down if enough experience is removed. Experience cannot go below 0 and level cannot go below 1.

```
void RemoveExp(long amount)
```

Parameters

amount long

The amount of experience to remove.

ResetToLevelOne()

Resets the entity to level 1 with 0 experience. This will trigger OnLevelDown events for each level lost.

```
void ResetToLevelOne()
```

SetTotalCurrentExp(long)

Sets the total current experience and updates the level accordingly. The level will be recalculated based on the new experience total.

```
void SetTotalCurrentExp(long totalCurrentExperience)
```

Parameters

totalCurrentExperience long

The total experience to set.

ValidateExperience()

Validates that the current experience matches the current level. Used for debugging and ensuring data consistency.

```
void ValidateExperience()
```


Namespace ElectricDrill.AstraRpgFramework. GameActions

Classes

[GameActionRunner](#)

MonoBehaviour that manages fire-and-forget execution of GameActions with automatic lifecycle management. Tracks running actions and cancels them when the GameObject is destroyed. Use `GameAction.RunFireAndForget()` which automatically manages this component.

[GameAction<TContext>](#)

Abstract base class for executable actions that can be configured as ScriptableObject assets. Derive from this class to create reusable, serializable actions that can be assigned in the editor. Uses Unity's Awaitable system for zero-allocation async execution.

Interfaces

[IExecutable<TContext>](#)

Defines a contract for executable actions using Unity's Awaitable system. This interface supports the Strategy pattern with async execution.

Class GameActionRunner

Namespace: [ElectricDrill.AstraRpgFramework.GameActions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

MonoBehaviour that manages fire-and-forget execution of GameActions with automatic lifecycle management. Tracks running actions and cancels them when the GameObject is destroyed. Use GameAction.RunFireAndForget() which automatically manages this component.

```
[DisallowMultipleComponent]
```

```
public class GameActionRunner : MonoBehaviour
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← GameActionRunner

Properties

RunningCount

Gets the number of currently running actions.

```
public int RunningCount { get; }
```

Property Value

int

Methods

CancelAll()

Cancels all running actions immediately.

```
public void CancelAll()
```

Run<TContext>(GameAction<TContext>, TContext)

Executes a GameAction fire-and-forget with automatic cancellation tracking. Multiple actions can run concurrently.

```
public void Run<TContext>(GameAction<TContext> action, TContext context)
```

Parameters

action [GameAction](#)<TContext>

The action to execute.

context TContext

The context to pass to the action.

Type Parameters

TContext

The context type for the action.

Class `GameAction<TContext>`

Namespace: [ElectricDrill.AstraRpgFramework.GameActions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Abstract base class for executable actions that can be configured as `ScriptableObject` assets. Derive from this class to create reusable, serializable actions that can be assigned in the editor. Uses Unity's Awaitable system for zero-allocation async execution.

```
public abstract class GameAction<TContext> : ScriptableObject, IExecutable<TContext>
```

Type Parameters

`TContext`

The type of context object passed to the execution method.

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← `GameAction<TContext>`

Implements

[IExecutable](#)<TContext>

Derived

[CompositeGameAction<TContext>](#), [DelayedGameAction<TContext>](#),
[DestroyGameObjectGameAction<TContext>](#), [DoNothingGameAction<TContext>](#),
[IncreaseCounterGameAction<TContext>](#), [ToggleActiveGameObjectGameAction<TContext>](#),
[ToggleCollidersGameAction<TContext>](#), [ToggleHarvestedExpSourceGameAction<TContext>](#),
[ToggleRenderersGameAction<TContext>](#)

Properties

DisplayName

A human-readable name for this action, shown in editors and debugging. Override to provide a descriptive name for your action.

```
public virtual string DisplayName { get; }
```

Property Value

string

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public abstract Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken = default)
```

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

ExecuteAsyncForUnityEvent(TContext)

Wrapper to execute this action asynchronously for UnityEvent calls. Runs in fire-and-forget manner. Exceptions are caught and logged to avoid unhandled exceptions in UnityEvent calls.

```
public void ExecuteAsyncForUnityEvent(TContext context)
```

Parameters

context TContext

The context to pass to the action.

OnOperationCanceled()

```
protected virtual void OnOperationCanceled()
```

RunFireAndForget(TContext, MonoBehaviour)

Executes this action fire-and-forget with automatic lifecycle management. The action will be canceled if the owner GameObject is destroyed. Multiple actions can run concurrently on the same owner.

```
public void RunFireAndForget(TContext context, MonoBehaviour owner)
```

Parameters

context TContext

The context to pass to the action.

owner [MonoBehaviour](#)

The MonoBehaviour that owns this action's lifecycle. A GameActionRunner will be automatically added to its GameObject if needed.

Interface IExecutable<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Defines a contract for executable actions using Unity's Awaitable system. This interface supports the Strategy pattern with async execution.

```
public interface IExecutable<in TContext>
```

Type Parameters

TContext

The type of context object passed to the execution method.

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context.

```
Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken = default)
```

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation.

Returns

[Awaitable](#) 

An Awaitable that completes when the action finishes.

Namespace ElectricDrill.AstraRpgFramework. GameActions.Actions

Classes

[CompositeGameAction<TContext>](#)

Executes a list of actions in sequence. Useful for chaining multiple behaviors (e.g., play sound, wait, disable object). Will gracefully terminate if the context is destroyed or cancellation is requested.

[DelayedGameAction<TContext>](#)

Waits for a specified duration before executing another action.

[DestroyGameObjectGameAction<TContext>](#)

Base action to destroy a Component's GameObject.

[DoNothingGameAction<TContext>](#)

An action that does nothing. Useful as a placeholder or default value.

[IncreaseCounterGameAction<TContext>](#)

[ToggleActiveGameObjectGameAction<TContext>](#)

Base action to set the active state of a Component's GameObject.

[ToggleCollidersGameAction<TContext>](#)

Base action to enable/disable all Colliders on a Component's GameObject.

[ToggleHarvestedExpSourceGameAction<TContext>](#)

Base generic game action for toggling the harvested state of ExpSource components. Sets the Harvested flag based on the `_enable` field, allowing control over whether the experience source can be collected.

[ToggleRenderersGameAction<TContext>](#)

Base action to enable/disable all Renderers on a Component's GameObject.

Class CompositeGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Executes a list of actions in sequence. Useful for chaining multiple behaviors (e.g., play sound, wait, disable object). Will gracefully terminate if the context is destroyed or cancellation is requested.

```
public abstract class CompositeGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext>
```

Type Parameters

TContext

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<TContext> ← CompositeGameAction<TContext>

Implements

[IExecutable](#)<TContext>

Derived

[CompositeComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken
    = default)
```

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

Class DelayedGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Waits for a specified duration before executing another action.

```
public abstract class DelayedGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext>
```

Type Parameters

TContext

Inheritance

object < [Object](#) < [ScriptableObject](#) < [GameAction](#) <TContext> < DelayedGameAction<TContext>

Implements

[IExecutable](#) <TContext>

Derived

[DelayedComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken
    = default)
```

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

Class

DestroyGameObjectGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Base action to destroy a Component's GameObject.

```
public abstract class DestroyGameObjectGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext> where TContext : Component
```

Type Parameters

TContext

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<TContext> ←
DestroyGameObjectGameAction<TContext>

Implements

[IExecutable](#)<TContext>

Derived

[DestroyGameObjectComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken)
```

= default)

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

Class DoNothingGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

An action that does nothing. Useful as a placeholder or default value.

```
public abstract class DoNothingGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext>
```

Type Parameters

TContext

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<TContext> ← DoNothingGameAction<TContext>

Implements

[IExecutable](#)<TContext>

Derived

[DoNothingComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken
    = default)
```

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

Class IncreaseCounterGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
public class IncreaseCounterGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext>
```

Type Parameters

TContext

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<TContext> ←
IncreaseCounterGameAction<TContext>

Implements

[IExecutable](#)<TContext>

Derived

[IncreaseCounterComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Properties

Counter

The counter to increase.

```
public LongRef Counter { get; set; }
```

Property Value

[LongRef](#)

IncreaseBy

The amount to increase the counter by. Use negative values to decrease.

```
[Tooltip("The amount to increase the counter by. Use negative values to decrease.")]  
public int IncreaseBy { get; set; }
```

Property Value

int

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext _, CancellationToken cancellationToken  
= default)
```

Parameters

_ TContext

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

Class

ToggleActiveGameObjectGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Base action to set the active state of a Component's GameObject.

```
public abstract class ToggleActiveGameObjectGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext> where TContext : Component
```

Type Parameters

TContext

Inheritance

object ← [Object](#) ↗ ← [ScriptableObject](#) ↗ ← [GameAction](#) <TContext> ←
ToggleActiveGameObjectGameAction<TContext>

Implements

[IExecutable](#) <TContext>

Derived

[ToggleActiveGameObjectComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Fields

_isActive

```
[SerializeField]  
[Tooltip("The active state to apply.")]
```

```
protected bool _isActive
```

Field Value

bool

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken  
= default)
```

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

Class ToggleCollidersGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Base action to enable/disable all Colliders on a Component's GameObject.

```
public abstract class ToggleCollidersGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext> where TContext : Component
```

Type Parameters

TContext

Inheritance

object < [Object](#) < [ScriptableObject](#) < [GameAction](#) <TContext> < ToggleCollidersGameAction<TContext>

Implements

[IExecutable](#)<TContext>

Derived

[ToggleCollidersComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Fields

_enable

```
[SerializeField]
[Tooltip("Function to apply to colliders (True = Enable, False = Disable)")]
protected bool _enable
```

Field Value

bool

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken  
= default)
```

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

Class

ToggleHarvestedExpSourceGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Base generic game action for toggling the harvested state of ExpSource components. Sets the Harvested flag based on the `_enable` field, allowing control over whether the experience source can be collected.

```
public abstract class ToggleHarvestedExpSourceGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext> where TContext : Component
```

Type Parameters

TContext

The context type that must be a Component with ExpSource.

Inheritance

object $\leftarrow$ [Object](#) $\leftarrow$ [ScriptableObject](#) $\leftarrow$ [GameAction](#)<TContext> $\leftarrow$
ToggleHarvestedExpSourceGameAction<TContext>

Implements

[IExecutable](#)<TContext>

Derived

[ToggleHarvestedExpSourceComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the toggle harvest logic asynchronously. Attempts to get ExpSource from the context and sets its Harvested flag based on _enable.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken  
= default)
```

Parameters

context TContext

The context component, expected to have or be ExpSource.

cancellationToken CancellationToken

Token to cancel the operation.

Returns

[Awaitable](#)

An Awaitable that completes when the toggle operation finishes.

Class ToggleRenderersGameAction<TContext>

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Base action to enable/disable all Renderers on a Component's GameObject.

```
public abstract class ToggleRenderersGameAction<TContext> : GameAction<TContext>,
    IExecutable<TContext> where TContext : Component
```

Type Parameters

TContext

Inheritance

object < [Object](#) < [ScriptableObject](#) < [GameAction](#) <TContext> < ToggleRenderersGameAction<TContext>

Implements

[IExecutable](#)<TContext>

Derived

[ToggleRenderersComponentGameAction](#)

Inherited Members

[GameAction<TContext>.DisplayName](#) , [GameAction<TContext>.ExecuteAsyncForUnityEvent\(TContext\)](#) ,
[GameAction<TContext>.OnOperationCanceled\(\)](#) ,
[GameAction<TContext>.RunFireAndForget\(TContext, MonoBehaviour\)](#)

Fields

_enable

```
[SerializeField]
[Tooltip("Function to apply to renderers (True = Show, False = Hide)")]
protected bool _enable
```

Field Value

bool

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken  
= default)
```

Parameters

context TContext

The context object containing information needed for execution.

cancellationToken CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

[Awaitable](#)

An Awaitable that completes when the action finishes.

Namespace ElectricDrill.AstraRpgFramework. GameActions.Actions.Component Classes

[CompositeComponentGameAction](#)

[DelayedComponentGameAction](#)

[DestroyGameObjectComponentGameAction](#)

[DoNothingComponentGameAction](#)

[IncreaseCounterComponentGameAction](#)

[ToggleActiveGameObjectComponentGameAction](#)

[ToggleCollidersComponentGameAction](#)

[ToggleHarvestedExpSourceComponentGameAction](#)

A game action used to toggle the harvested state of an ExpSource component. Sets the Harvested flag based on the enable setting, controlling whether the experience can be collected. Provided as a ScriptableObject for easy assignment in the editor. Create instances via Assets -> Create -> Astra RPG Framework / Game Actions / Context: Component / Toggle Harvest Exp Source.

[ToggleRenderersComponentGameAction](#)

Class CompositeComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
[CreateAssetMenu(fileName = "New Composite Component Action", menuName = "Astra RPG Framework/Game Actions/Context: Component/Composite", order = 10)]  
public class CompositeComponentGameAction : CompositeGameAction<Component>,  
IExecutable<Component>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<[Component](#)> ← [CompositeGameAction](#)<[Component](#)> ← CompositeComponentGameAction

Implements

[IExecutable](#)<[Component](#)>

Inherited Members

[CompositeGameAction](#)<[Component](#)>.ExecuteAsync([Component](#), [CancellationToken](#)) ,
[GameAction](#)<[Component](#)>.DisplayName ,
[GameAction](#)<[Component](#)>.ExecuteAsyncForUnityEvent([Component](#)) ,
[GameAction](#)<[Component](#)>.OnOperationCanceled() ,
[GameAction](#)<[Component](#)>.RunFireAndForget([Component](#), [MonoBehaviour](#))

Class DelayedComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
[CreateAssetMenu(fileName = "New Delayed Component Action", menuName = "Astra RPG Framework/Game Actions/Context: Component/Delayed", order = 10)]  
public class DelayedComponentGameAction : DelayedGameAction<Component>, IExecutable<Component>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<[Component](#)> ← [DelayedGameAction](#)<[Component](#)> ← DelayedComponentGameAction

Implements

[IExecutable](#)<[Component](#)>

Inherited Members

[DelayedGameAction](#)<[Component](#)>.ExecuteAsync([Component](#), [CancellationToken](#)) ,
[GameAction](#)<[Component](#)>.DisplayName ,
[GameAction](#)<[Component](#)>.ExecuteAsyncForUnityEvent([Component](#)) ,
[GameAction](#)<[Component](#)>.OnOperationCanceled() ,
[GameAction](#)<[Component](#)>.RunFireAndForget([Component](#), [MonoBehaviour](#))

Class

DestroyGameObjectComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
[CreateAssetMenu(fileName = "New Destroy GO Component Action", menuName = "Astra RPG Framework/Game Actions/Context: Component/Destroy Game Object", order = 10)]  
public class DestroyGameObjectComponentGameAction : DestroyGameObjectGameAction<Component>, IExecutable<Component>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<[Component](#)> ← [DestroyGameObjectGameAction](#)<[Component](#)> ← DestroyGameObjectComponentGameAction

Implements

[IExecutable](#)<[Component](#)>

Inherited Members

[DestroyGameObjectGameAction](#)<[Component](#)>.ExecuteAsync([Component](#), [CancellationToken](#)) ,
[GameAction](#)<[Component](#)>.DisplayName ,
[GameAction](#)<[Component](#)>.ExecuteAsyncForUnityEvent([Component](#)) ,
[GameAction](#)<[Component](#)>.OnOperationCanceled() ,
[GameAction](#)<[Component](#)>.RunFireAndForget([Component](#), [MonoBehaviour](#))

Class DoNothingComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
[CreateAssetMenu(fileName = "New Do Nothing Component Action", menuName = "Astra RPG  
Framework/Game Actions/Context: Component/Do Nothing", order = 10)]  
public class DoNothingComponentGameAction : DoNothingGameAction<Component>,  
IExecutable<Component>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<[Component](#)> ←
[DoNothingGameAction](#)<[Component](#)> ← DoNothingComponentGameAction

Implements

[IExecutable](#)<[Component](#)>

Inherited Members

[DoNothingGameAction](#)<[Component](#)>.ExecuteAsync([Component](#), [CancellationToken](#)),
[GameAction](#)<[Component](#)>.DisplayName ,
[GameAction](#)<[Component](#)>.ExecuteAsyncForUnityEvent([Component](#)) ,
[GameAction](#)<[Component](#)>.OnOperationCanceled() ,
[GameAction](#)<[Component](#)>.RunFireAndForget([Component](#), [MonoBehaviour](#)).

Class IncreaseCounterComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
[CreateAssetMenu(fileName = "New Increase Counter Component Game Action", menuName = "Astra  
RPG Framework/Game Actions/Context: Component/Increase Counter", order = 11)]  
public class IncreaseCounterComponentGameAction : IncreaseCounterGameAction<Component>,  
IExecutable<Component>
```

Inheritance

object < [Object](#) < [ScriptableObject](#) < [GameAction](#) < [Component](#) > < [IncreaseCounterGameAction](#) < [Component](#) > < IncreaseCounterComponentGameAction

Implements

[IExecutable](#) < [Component](#) >

Inherited Members

[IncreaseCounterGameAction](#) < [Component](#) > .Counter ,
[IncreaseCounterGameAction](#) < [Component](#) > .IncreaseBy ,
[IncreaseCounterGameAction](#) < [Component](#) > .ExecuteAsync([Component](#), [CancellationToken](#)) ,
[GameAction](#) < [Component](#) > .DisplayName ,
[GameAction](#) < [Component](#) > .ExecuteAsyncForUnityEvent([Component](#)) ,
[GameAction](#) < [Component](#) > .OnOperationCanceled() ,
[GameAction](#) < [Component](#) > .RunFireAndForget([Component](#), [MonoBehaviour](#)) .

Class

ToggleActiveGameObjectComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
[CreateAssetMenu(fileName = "New Toggle Active GO Component Action", menuName = "Astra RPG Framework/Game Actions/Context: Component/Toggle Active GO", order = 10)]  
public class ToggleActiveGameObjectComponentGameAction :  
    ToggleActiveGameObjectGameAction<Component>, IExecutable<Component>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<[Component](#)> ←
[ToggleActiveGameObjectGameAction](#)<[Component](#)> ←
[ToggleActiveGameObjectComponentGameAction](#)

Implements

[IExecutable](#)<[Component](#)>

Inherited Members

[ToggleActiveGameObjectGameAction](#)<[Component](#)>.IsActive ,
[ToggleActiveGameObjectGameAction](#)<[Component](#)>.ExecuteAsync([Component](#), [CancellationToken](#)) ,
[GameAction](#)<[Component](#)>.DisplayName ,
[GameAction](#)<[Component](#)>.ExecuteAsyncForUnityEvent([Component](#)) ,
[GameAction](#)<[Component](#)>.OnOperationCanceled() ,
[GameAction](#)<[Component](#)>.RunFireAndForget([Component](#), [MonoBehaviour](#))

Class ToggleCollidersComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
[CreateAssetMenu(fileName = "New Toggle Colliders Component Action", menuName = "Astra RPG Framework/Game Actions/Context: Component/Toggle Colliders", order = 10)]  
public class ToggleCollidersComponentGameAction : ToggleCollidersGameAction<Component>, IExecutable<Component>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<[Component](#)> ←

[ToggleCollidersGameAction](#)<[Component](#)> ← ToggleCollidersComponentGameAction

Implements

[IExecutable](#)<[Component](#)>

Inherited Members

[ToggleCollidersGameAction](#)<[Component](#)>.[enable](#) ,

[ToggleCollidersGameAction](#)<[Component](#)>.[ExecuteAsync](#)([Component](#), [CancellationToken](#)) ,

[GameAction](#)<[Component](#)>.[DisplayName](#) ,

[GameAction](#)<[Component](#)>.[ExecuteAsyncForUnityEvent](#)([Component](#)) ,

[GameAction](#)<[Component](#)>.[OnOperationCanceled](#)() ,

[GameAction](#)<[Component](#)>.[RunFireAndForget](#)([Component](#), [MonoBehaviour](#))

Class

ToggleHarvestedExpSourceComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A game action used to toggle the harvested state of an ExpSource component. Sets the Harvested flag based on the enable setting, controlling whether the experience can be collected. Provided as a ScriptableObject for easy assignment in the editor. Create instances via Assets -> Create -> Astra RPG Framework / Game Actions / Context: Component / Toggle Harvest Exp Source.

```
[CreateAssetMenu(fileName = "Toggle Harvested Exp Source Game Action", menuName = "Astra RPG Framework/Game Actions/Context: Component/Toggle Harvested Exp Source")]  
public class ToggleHarvestedExpSourceComponentGameAction :  
    ToggleHarvestedExpSourceGameAction<Component>, IExecutable<Component>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<[Component](#)> ←
[ToggleHarvestedExpSourceGameAction](#)<[Component](#)> ←
[ToggleHarvestedExpSourceComponentGameAction](#)

Implements

[IExecutable](#)<[Component](#)>

Inherited Members

[ToggleHarvestedExpSourceGameAction](#)<[Component](#)>.ExecuteAsync([Component](#), [CancellationToken](#)),
[GameAction](#)<[Component](#)>.DisplayName ,
[GameAction](#)<[Component](#)>.ExecuteAsyncForUnityEvent([Component](#)) ,
[GameAction](#)<[Component](#)>.OnOperationCanceled() ,
[GameAction](#)<[Component](#)>.RunFireAndForget([Component](#), [MonoBehaviour](#)).

Class ToggleRenderersComponentGameAction

Namespace: [ElectricDrill.AstraRpgFramework.GameActions.Actions.Component](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
[CreateAssetMenu(fileName = "New Toggle Renderers Component Action", menuName = "Astra RPG Framework/Game Actions/Context: Component/Toggle Renderers", order = 10)]  
public class ToggleRenderersComponentGameAction : ToggleRenderersGameAction<Component>, IExecutable<Component>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [GameAction](#)<[Component](#)> ← [ToggleRenderersGameAction](#)<[Component](#)> ← ToggleRenderersComponentGameAction

Implements

[IExecutable](#)<[Component](#)>

Inherited Members

[ToggleRenderersGameAction](#)<[Component](#)>.enable ,
[ToggleRenderersGameAction](#)<[Component](#)>.ExecuteAsync([Component](#), [CancellationToken](#)) ,
[GameAction](#)<[Component](#)>.DisplayName ,
[GameAction](#)<[Component](#)>.ExecuteAsyncForUnityEvent([Component](#)) ,
[GameAction](#)<[Component](#)>.OnOperationCanceled() ,
[GameAction](#)<[Component](#)>.RunFireAndForget([Component](#), [MonoBehaviour](#))

Namespace ElectricDrill.AstraRpgFramework.

Scaling

Classes

[ScalingComponent](#)

Abstract base class for all scaling components. A scaling component calculates a value based on an entity's properties.

[ScalingFormula](#)

Represents a formula to calculate a value based on values returned by several scaling components. The calculation of each scaling component can either be based on the "self" entity (the entity that owns the formula) or the "target" entity (the entity that will be targeted by the action). For example, a scaling formula for an attack that deals the more damage the higher the enemy's armor is, can use the "self" entity to calculate the damage based on the attacker's stats and the "target" entity to calculate additional damage based on the target's armor. The final damage will be the sum of the value returned by the two scaling components.

[ScalingFormulaInstance](#)

A per-entity runtime wrapper around a shared [ScalingFormula](#) asset.

Because [ScalingFormula](#) is a `ScriptableObject`, it is shared across all entities that reference it. Mutating temporary scaling components directly on the asset would therefore affect every entity that uses it. [ScalingFormulaInstance](#) solves this by keeping its own temporary component lists while delegating all computation to the underlying asset.

Each entity that needs per-entity runtime modifications (e.g. a buff that adds an extra scaling term) should create its own [ScalingFormulaInstance](#) from the shared [ScalingFormula](#) asset.

[SoSetScalingComponentBase<SetType, KeyType>](#)

Abstract base class for scaling components that work with `ScriptableObject`-based sets.

Interfaces

[IEntityValueCalculator](#)

[IScalingFormula](#)

Defines the contract for evaluating a scaling formula against one or two entities.

Implementations include:

- [ScalingFormula](#) – a shared `ScriptableObject` asset with no per-entity state.
- [ScalingFormulaInstance](#) – a per-entity runtime wrapper that supports temporary scaling components without affecting other entities that reference the same [ScalingFormula](#).

Interface IEntityValueCalculator

Namespace: [ElectricDrill.AstraRpgFramework.Scaling](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

```
public interface IEntityValueCalculator
```

Methods

CalculateValue(EntityCore)

```
long CalculateValue(EntityCore entity)
```

Parameters

entity [EntityCore](#)

Returns

long

Interface IScalingFormula

Namespace: [ElectricDrill.AstraRpgFramework.Scaling](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Defines the contract for evaluating a scaling formula against one or two entities.

Implementations include:

- [ScalingFormula](#) – a shared `ScriptableObject` asset with no per-entity state.
- [ScalingFormulaInstance](#) – a per-entity runtime wrapper that supports temporary scaling components without affecting other entities that reference the same [ScalingFormula](#).

```
public interface IScalingFormula
```

Methods

CalculateValue(EntityCore)

Calculates the final value using only the "self" entity. Assumes the formula has no target-scaling components and uses a fixed base value.

```
long CalculateValue(EntityCore self)
```

Parameters

`self` [EntityCore](#)

Returns

long

CalculateValue(EntityCore, EntityCore)

Calculates the final value using both "self" and "target" entities. Assumes the formula uses a fixed base value.

```
long CalculateValue(EntityCore self, EntityCore target)
```

Parameters

self [EntityCore](#)

target [EntityCore](#)

Returns

long

CalculateValue(EntityCore, EntityCore, int)

Calculates the final value using both "self" and "target" entities and a specific level for the base value. Assumes the formula uses a growth-based base value.

```
long CalculateValue(EntityCore self, EntityCore target, int level)
```

Parameters

self [EntityCore](#)

target [EntityCore](#)

level int

Returns

long

CalculateValue(EntityCore, int)

Calculates the final value using only the "self" entity and a specific level for the base value. Assumes the formula has no target-scaling components and uses a growth-based base value.

```
long CalculateValue(EntityCore self, int level)
```

Parameters

`self` [EntityCore](#)

`level` int

Returns

long

Class ScalingComponent

Namespace: [ElectricDrill.AstraRpgFramework.Scaling](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Abstract base class for all scaling components. A scaling component calculates a value based on an entity's properties.

```
public abstract class ScalingComponent : ScriptableObject, IEntityValueCalculator
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← ScalingComponent

Implements

[IEntityValueCalculator](#)

Derived

[SoSetScalingComponentBase<SetType, KeyType>](#)

Methods

CalculateValue(EntityCore)

Calculates the value of this component based on the provided entity.

```
public abstract long CalculateValue(EntityCore entity)
```

Parameters

entity [EntityCore](#)

The entity whose properties are used for calculation.

Returns

long

The calculated value from this component.

Class ScalingFormula

Namespace: [ElectricDrill.AstraRpgFramework.Scaling](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents a formula to calculate a value based on values returned by several scaling components. The calculation of each scaling component can either be based on the "self" entity (the entity that owns the formula) or the "target" entity (the entity that will be targeted by the action). For example, a scaling formula for an attack that deals the more damage the higher the enemy's armor is, can use the "self" entity to calculate the damage based on the attacker's stats and the "target" entity to calculate additional damage based on the target's armor. The final damage will be the sum of the value returned by the two scaling components.

```
public class ScalingFormula : ScriptableObject, IScalingFormula, IEntityValueCalculator
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← ScalingFormula

Implements

[IScalingFormula](#), [IEntityValueCalculator](#)

Properties

HasSelfComponents

Whether this formula has any serialized self-scaling components.

```
public bool HasSelfComponents { get; }
```

Property Value

bool

HasTargetComponents

Whether this formula has any serialized target-scaling components.

```
public bool HasTargetComponents { get; }
```

Property Value

bool

SelfScalingComponents

Read-only view of the serialized self-scaling components.

```
public IReadOnlyList<ScalingComponent> SelfScalingComponents { get; }
```

Property Value

IReadOnlyList<[ScalingComponent](#)>

TargetScalingComponents

Read-only view of the serialized target-scaling components.

```
public IReadOnlyList<ScalingComponent> TargetScalingComponents { get; }
```

Property Value

IReadOnlyList<[ScalingComponent](#)>

TmpSelfScalingComponents

Gets the list of temporary scaling components for the "self" entity.

```
[Obsolete("Mutating temporary components on a shared ScriptableObject affects all entities  
that reference it. Create a ScalingFormulaInstance per entity and use its  
TmpSelfScalingComponents instead.")]  
public List<ScalingComponent> TmpSelfScalingComponents { get; }
```

Property Value

List<[ScalingComponent](#)>

Remarks

Deprecated. Because [ScalingFormula](#) is a shared `ScriptableObject`, mutating this list affects *every* entity that references the same asset. Use [TmpSelfScalingComponents](#) instead to keep temporary components isolated per entity.

TmpTargetScalingComponents

Gets the list of temporary scaling components for the "target" entity.

```
[Obsolete("Mutating temporary components on a shared ScriptableObject affects all entities  
that reference it. Create a ScalingFormulaInstance per entity and use its  
TmpTargetScalingComponents instead.")]  
public List<ScalingComponent> TmpTargetScalingComponents { get; }
```

Property Value

List<[ScalingComponent](#)>

Remarks

Deprecated. Because [ScalingFormula](#) is a shared `ScriptableObject`, mutating this list affects *every* entity that references the same asset. Use [TmpTargetScalingComponents](#) instead to keep temporary components isolated per entity.

UseScalingBaseValue

Whether this formula calculates its base value from a GrowthFormula at a given level.

```
public bool UseScalingBaseValue { get; }
```

Property Value

bool

Methods

CalculateSelfScalingComponents(EntityCore)

```
public long CalculateSelfScalingComponents(EntityCore self)
```

Parameters

self [EntityCore](#)

Returns

long

CalculateTargetScalingComponents(EntityCore)

```
public long CalculateTargetScalingComponents(EntityCore target)
```

Parameters

target [EntityCore](#)

Returns

long

CalculateValue(EntityCore)

Calculates the final value using only the "self" entity. This method assumes there are no target scaling components.

```
public long CalculateValue(EntityCore self)
```

Parameters

self [EntityCore](#)

The entity providing values for self-scaling components.

Returns

long

The calculated value.

CalculateValue(EntityCore, EntityCore)

Calculates the final value using both "self" and "target" entities.

```
public long CalculateValue(EntityCore self, EntityCore target)
```

Parameters

self [EntityCore](#)

The entity providing values for self-scaling components.

target [EntityCore](#)

The entity providing values for target-scaling components.

Returns

long

The calculated value.

CalculateValue(EntityCore, EntityCore, int)

Calculates the final value using both "self" and "target" entities, and a specific level for the base value. This method assumes the formula uses a scaling base value.

```
public long CalculateValue(EntityCore self, EntityCore target, int level)
```

Parameters

self [EntityCore](#)

The entity providing values for self-scaling components.

target [EntityCore](#)

The entity providing values for target-scaling components.

level int

The level to use for calculating the base value.

Returns

long

The calculated value.

CalculateValue(EntityCore, int)

Calculates the final value using the "self" entity and a specific level for the base scaling value. This method assumes there are no target scaling components and the formula uses a scaling base value.

```
public long CalculateValue(EntityCore self, int level)
```

Parameters

self [EntityCore](#)

The entity providing values for self-scaling components.

level int

The level to use for calculating the base value.

Returns

long

The calculated value.

GetBaseValue()

Returns the fixed base value of the formula. If `useScalingBaseValue` is true, an exception is thrown because a level is required.

```
public long GetBaseValue()
```

Returns

long

The fixed base value.

Exceptions

[AssertionException](#)[↗]

Thrown if `useScalingBaseValue` is true, since a level must be specified in that case.

GetBaseValue(int)

Returns the base value of the formula for the specified level. If `useScalingBaseValue` is true, the value is calculated using the associated growth formula. If `useScalingBaseValue` is false and a level is provided, an exception is thrown.

```
public long GetBaseValue(int level)
```

Parameters

`level` int

The level to use for calculating the base value.

Returns

long

The calculated base value.

Exceptions

Thrown if the growth formula is not set, if the level is less than or equal to zero, if the level exceeds the maximum allowed by the growth formula, or if `useScalingBaseValue` is false and a level is provided.

ResetTmpScalings()

Clears all temporary scaling components for both "self" and "target".

```
[Obsolete("Resetting temporary components on a shared ScriptableObject affects all entities  
that reference it. Create a ScalingFormulaInstance per entity and call ResetTmpScalings on  
the instance instead.")]  
public void ResetTmpScalings()
```

Remarks

Deprecated. Because [ScalingFormula](#) is a shared `ScriptableObject`, this call clears temporary components for *every* entity that references the same asset. Use [ResetTmpScalings\(\)](#) instead.

Class ScalingFormulaInstance

Namespace: [ElectricDrill.AstraRpgFramework.Scaling](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A per-entity runtime wrapper around a shared [ScalingFormula](#) asset.

Because [ScalingFormula](#) is a `ScriptableObject`, it is shared across all entities that reference it. Mutating temporary scaling components directly on the asset would therefore affect every entity that uses it. [ScalingFormulaInstance](#) solves this by keeping its own temporary component lists while delegating all computation to the underlying asset.

Each entity that needs per-entity runtime modifications (e.g. a buff that adds an extra scaling term) should create its own [ScalingFormulaInstance](#) from the shared [ScalingFormula](#) asset.

```
public class ScalingFormulaInstance : IScalingFormula
```

Inheritance

object ← ScalingFormulaInstance

Implements

[IScalingFormula](#)

Examples

```
// At ability / buff initialisation:
var instance = new ScalingFormulaInstance(myScalingFormulaAsset);

// When a buff is applied to this specific entity:
instance.TmpSelfScalingComponents.Add(armorScalingComponent);

// When the buff expires:
instance.TmpSelfScalingComponents.Remove(armorScalingComponent);
// - or reset all temporaries at once:
instance.ResetTmpScalings();
```

Constructors

ScalingFormulaInstance(ScalingFormula)

```
public ScalingFormulaInstance(ScalingFormula formula)
```

Parameters

`formula` [ScalingFormula](#)

The shared [ScalingFormula](#) asset this instance wraps.

Properties

TmpSelfScalingComponents

Per-entity temporary scaling components applied to the "self" entity.

```
public List<ScalingComponent> TmpSelfScalingComponents { get; }
```

Property Value

List<[ScalingComponent](#)>

TmpTargetScalingComponents

Per-entity temporary scaling components applied to the "target" entity.

```
public List<ScalingComponent> TmpTargetScalingComponents { get; }
```

Property Value

List<[ScalingComponent](#)>

Methods

CalculateValue(EntityCore)

Calculates the final value using only the "self" entity. Assumes the formula has no target-scaling components and uses a fixed base value.

```
public long CalculateValue(EntityCore self)
```

Parameters

self [EntityCore](#)

Returns

long

CalculateValue(EntityCore, EntityCore)

Calculates the final value using both "self" and "target" entities. Assumes the formula uses a fixed base value.

```
public long CalculateValue(EntityCore self, EntityCore target)
```

Parameters

self [EntityCore](#)

target [EntityCore](#)

Returns

long

CalculateValue(EntityCore, EntityCore, int)

Calculates the final value using both "self" and "target" entities and a specific level for the base value. Assumes the formula uses a growth-based base value.

```
public long CalculateValue(EntityCore self, EntityCore target, int level)
```

Parameters

self [EntityCore](#)

target [EntityCore](#)

level int

Returns

long

CalculateValue(EntityCore, int)

Calculates the final value using only the "self" entity and a specific level for the base value. Assumes the formula has no target-scaling components and uses a growth-based base value.

```
public long CalculateValue(EntityCore self, int level)
```

Parameters

self [EntityCore](#)

level int

Returns

long

ResetTmpScalings()

Clears all per-entity temporary scaling components for both "self" and "target".

```
public void ResetTmpScalings()
```

Class SoSetScalingComponentBase<SetType, KeyType>

Namespace: [ElectricDrill.AstraRpgFramework.Scaling](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Abstract base class for scaling components that work with ScriptableObject-based sets.

```
public abstract class SoSetScalingComponentBase<SetType, KeyType> : ScalingComponent,
    IEntityValueCalculator, IValueProvider<KeyType, double> where SetType : ScriptableObject
```

Type Parameters

SetType

The type of ScriptableObject set that contains the items used for defining the scaling value

KeyType

The type of items within the set that can be used to calculate the scaling value

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [ScalingComponent](#) ←
SoSetScalingComponentBase<SetType, KeyType>

Implements

[IEntityValueCalculator](#), [IValueProvider](#)<KeyType, double>

Derived

[AttributesScalingComponent](#), [StatsScalingComponent](#)

Fields

_set

```
[SerializeField]  
protected SetType _set
```

Field Value

SetType

Properties

Set

The base set used for scaling calculations.

```
public SetType Set { get; }
```

Property Value

SetType

Methods

CalculateValue(EntityCore)

Calculates the final scaled value based on the entity's values and the configured scaling multipliers.

```
public override long CalculateValue(EntityCore entity)
```

Parameters

entity [EntityCore](#)

The entity to calculate the scaled value for

Returns

long

The calculated scaled value as a long integer

Get(KeyType)

Gets the value associated with the specified key.

```
public virtual double Get(KeyType key)
```

Parameters

key KeyType

The key used to retrieve the value.

Returns

double

The value associated with the specified key.

GetEntitySet(EntityCore)

Gets the set associated with the specified entity. This method must be implemented by derived classes to define how to retrieve the entity's set.

```
protected abstract SetType GetEntitySet(EntityCore entity)
```

Parameters

entity [EntityCore](#)

The entity to get the set for

Returns

SetType

The set associated with the entity

GetEntityValue(EntityCore, KeyType)

Gets the current value of a specific key for the given entity. This method must be implemented by derived classes to define how to retrieve entity values.


```
protected abstract long GetEntityValue(EntityCore entity, KeyType key)
```

Parameters

entity [EntityCore](#)

The entity to get the value from

key KeyType

The specific key to get the value for

Returns

long

The current value of the specified key for the entity

GetSetItems()

Gets all items from the current set that can be used for scaling. This method must be implemented by derived classes to define which items are available for scaling.

```
protected abstract IEnumerable<KeyType> GetSetItems()
```

Returns

IEnumerable<KeyType>

An enumerable collection of all scalable items in the set

OnValidate()

Validates and refreshes the scaling attribute values based on the current set. Called automatically when the component is validated in the editor.

```
protected virtual void OnValidate()
```

ValidateEntitySet(EntityCore)

Validates that the entity's set is compatible with this scaling component's set. By default, requires exact equality. Derived classes can override for different validation logic. If the entity's set is null, logs a warning and skips validation.

```
protected virtual void ValidateEntitySet(EntityCore entity)
```

Parameters

entity [EntityCore](#)

The entity to validate

Namespace ElectricDrill.AstraRpgFramework. Scaling.ScalingComponents

Classes

[AttributesScalingComponent](#)

A scaling component that calculates a value based on an entity's attributes.

[StatsScalingComponent](#)

A scaling component that calculates a value based on an entity's stats.

Class AttributesScalingComponent

Namespace: [ElectricDrill.AstraRpgFramework.Scaling.ScalingComponents](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A scaling component that calculates a value based on an entity's attributes.

```
public class AttributesScalingComponent : SoSetScalingComponentBase<AttributeSet,
Attribute>, IEntityValueCalculator, IValueProvider<Attribute, double>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [ScalingComponent](#) ←
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)> ← AttributesScalingComponent

Implements

[IEntityValueCalculator](#), [IValueProvider](#)<[Attribute](#), double>

Inherited Members

[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.set ,
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.Set ,
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.CalculateValue([EntityCore](#)) ,
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.Get([Attribute](#)) ,
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.ValidateEntitySet([EntityCore](#)) ,
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.GetEntitySet([EntityCore](#)) ,
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.GetEntityValue([EntityCore](#), [Attribute](#)) ,
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.OnValidate() ,
[SoSetScalingComponentBase](#)<[AttributeSet](#), [Attribute](#)>.GetSetItems() ,
[ScalingComponent](#).CalculateValue([EntityCore](#))

Methods

GetEntitySet(EntityCore)

Gets the AttributeSet from the specified entity.

```
protected override AttributeSet GetEntitySet(EntityCore entity)
```

Parameters

entity [EntityCore](#)

The entity.

Returns

[AttributeSet](#)

The entity's AttributeSet, or null if the entity has no EntityAttributes component.

GetEntityValue(EntityCore, Attribute)

Gets the value of a specific attribute from the entity.

```
protected override long GetEntityValue(EntityCore entity, Attribute key)
```

Parameters

entity [EntityCore](#)

The entity.

key [Attribute](#)

The attribute to get the value of.

Returns

long

The value of the attribute, or 0 if the entity has no EntityAttributes component.

GetSetItems()

Gets the collection of attributes from the associated AttributeSet.

```
protected override IEnumerable<Attribute> GetSetItems()
```

Returns

IEnumerable<[Attribute](#)>

An enumerable of attributes.

Class StatsScalingComponent

Namespace: [ElectricDrill.AstraRpgFramework.Scaling.ScalingComponents](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A scaling component that calculates a value based on an entity's stats.

```
public class StatsScalingComponent : SoSetScalingComponentBase<StatSet, Stat>,
    IEntityValueCalculator, IValueProvider<Stat, double>
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [ScalingComponent](#) ←
[SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)> ← StatsScalingComponent

Implements

[IEntityValueCalculator](#), [IValueProvider](#)<[Stat](#), double>

Inherited Members

[SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.set, [SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.Set, [SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.CalculateValue([EntityCore](#)), [SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.Get([Stat](#)), [SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.ValidateEntitySet([EntityCore](#)), [SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.GetEntitySet([EntityCore](#)), [SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.GetEntityValue([EntityCore](#), [Stat](#)), [SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.OnValidate(), [SoSetScalingComponentBase](#)<[StatSet](#), [Stat](#)>.GetSetItems(), [ScalingComponent](#).CalculateValue([EntityCore](#)).

Methods

GetEntitySet(EntityCore)

Gets the StatSet from the specified entity.

```
protected override StatSet GetEntitySet(EntityCore entity)
```

Parameters

entity [EntityCore](#)

The entity.

Returns

[StatSet](#)

The entity's StatSet, or null if the entity has no EntityStats component.

GetEntityValue(EntityCore, Stat)

Gets the value of a specific stat from the entity.

```
protected override long GetEntityValue(EntityCore entity, Stat key)
```

Parameters

entity [EntityCore](#)

The entity.

key [Stat](#)

The stat to get the value of.

Returns

long

The value of the stat, or 0 if the entity has no EntityStats component.

GetSetItems()

Gets the collection of stats from the associated StatSet.

```
protected override IEnumerable<Stat> GetSetItems()
```

Returns

IEnumerable<[Stat](#)>

An enumerable of stats.

ValidateEntitySet(EntityCore)

Validates that the entity's StatSet is equal to or contains the scaling component's StatSet. This allows entities with StatSets that include the component's StatSet to be compatible. If the entity's StatSet is null, logs a warning and skips validation.

```
protected override void ValidateEntitySet(EntityCore entity)
```

Parameters

entity [EntityCore](#)

The entity to validate

Namespace ElectricDrill.AstraRpgFramework.

Stats

Classes

[EntityStats](#)

Component that manages the statistics of an entity in the game. It handles base stats, flat stat modifiers, stat to stat modifiers, and percentage stat modifiers.

Base stats can either be fixed or come from the entity's class (if one is available on the Game Object). When stats change because of a modifier of any kind, the assigned [StatChangedGameEvent](#) is raised.

[Stat](#)

Represents a stat in the RPG system that extends BoundedValue with attribute scaling capabilities. Stats can be scaled based on entity attributes and provide equality comparison based on name.

[StatSet](#)

A ScriptableObject that defines a collection of stats that can be used by entities. Implements IStatContainer to provide stat containment functionality.

[StatSetInstance](#)

Represents an instance of a StatSet with actual values for each stat. Implements IEnumerable and IStatContainer to provide collection functionality.

[StatToStatModifier](#)

A ScriptableObject that defines how one stat can scale upon another stat basing on a percentage. Used to create relationships between different stats in the RPG system.

Structs

[StatChangeInfo](#)

Contains information about a stat change event, including the entity, stat, and old/new values. Used for tracking and responding to stat modifications.

Interfaces

[IHasStatSet](#)

Interface for objects that provide a source of stat sets. Defines the contract for accessing a StatSet that contains stat definitions.

[IStatReader](#)

Read-only interface for querying stat values from an entity. Use [TryGet\(Stat, out long\)](#) as the safe access path; [Get](#) requires [Contains\(T\)](#) to be checked first.

Class EntityStats

Namespace: [ElectricDrill.AstraRpgFramework.Stats](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Component that manages the statistics of an entity in the game. It handles base stats, flat stat modifiers, stat to stat modifiers, and percentage stat modifiers.

Base stats can either be fixed or come from the entity's class (if one is available on the Game Object).

When stats change because of a modifier of any kind, the assigned [StatChangedGameEvent](#) is raised.

```
[DisallowMultipleComponent]
[RequireComponent(typeof(EntityCore))]
public class EntityStats : MonoBehaviour, IHasStatSet, IValueProvider<Stat, long>,
    IStatReader, IValueContainer<Stat>
```

Inheritance

object ← [Object](#) ← [Component](#) ← [Behaviour](#) ← [MonoBehaviour](#) ← EntityStats

Implements

[IHasStatSet](#), [IValueProvider](#)<[Stat](#), long>, [IStatReader](#), [IValueContainer](#)<[Stat](#)>

Fields

_entityClass

```
protected IClassSource _entityClass
```

Field Value

[IClassSource](#)

_flatModifiers

```
protected StatSetInstance _flatModifiers
```

Field Value

[StatSetInstance](#)

_percentageModifiers

```
protected StatSetInstance _percentageModifiers
```

Field Value

[StatSetInstance](#)

Properties

EntityClass

The class source of the entity. In most cases, this is the [EntityClass](#) component attached to the entity.

```
public IClassSource EntityClass { get; }
```

Property Value

[IClassSource](#)

EntityCore

The entity core associated with this stats component.

```
public EntityCore EntityCore { get; }
```

Property Value

[EntityCore](#)

OnStatChanged

Event raised when a stat changes due to a modifier.

```
public StatChangedGameEvent OnStatChanged { get; }
```

Property Value

[StatChangedGameEvent](#)

StatSet

The stat set used to calculate the entity's stats.

```
public virtual StatSet StatSet { get; }
```

Property Value

[StatSet](#)

If `_useBaseStatsFromClass` is true, it returns the stat set of the entity's class. Otherwise, it returns the fixed base stats stat set.

UseClassBaseStats

Gets or sets whether to use the base stats from the entity's class. If true, the base stats are taken from the [EntityClass](#). If false, the base stats are taken from the fixed base stats defined in this component.

```
public bool UseClassBaseStats { get; }
```

Property Value

bool

Methods

AddFlatModifier(Stat, long)

Adds a flat modifier to a stat.

```
public void AddFlatModifier(Stat stat, long value)
```

Parameters

stat [Stat](#)

The stat to add the flat modifier to.

value long

The value of the flat modifier.

AddPercentageModifier(Stat, Percentage)

Adds a [Percentage](#) modifier to a stat. Such modifiers consider the base value of the stat, the flat modifiers, and the stat-to-stat modifiers.

```
public void AddPercentageModifier(Stat stat, Percentage value)
```

Parameters

stat [Stat](#)

The stat to add the percentage modifier to.

value [Percentage](#)

The value of the percentage modifier.

AddStatToStatModifier(Stat, Stat, Percentage)

Adds a stat-to-stat modifier. Such modifiers add a percentage of the source stat to the target stat. Such modifiers consider the base value and the flat modifiers of the source stat.

```
public void AddStatToStatModifier(Stat target, Stat source, Percentage percentage)
```

Parameters

target [Stat](#)

The target stat.

source [Stat](#)

The source stat.

percentage [Percentage](#)

The [Percentage](#) of the source stat to add to the target stat.

Contains(Stat)

Returns whether `stat` is tracked by this component's stat set.

```
public bool Contains(Stat stat)
```

Parameters

`stat` [Stat](#)

Returns

bool

Get(Stat)

The final value of a stat, considering all the modifiers. Calculation is done in the following order:

1. Base value
2. Flat modifiers
3. Stat to stat modifiers
4. Percentage modifiers

```
public virtual long Get(Stat stat)
```

Parameters

`stat` [Stat](#)

The stat to get the final value of.

Returns

long

The final value of the stat if the component is enabled (clamped to min/max values); otherwise, 0

GetBase(Stat)

The base value is the value of the stat without any modifiers. If UseClassBaseStats is true, it returns the value from the entity's class. Otherwise, it returns the value from the fixed base stats.

```
public long GetBase(Stat stat)
```

Parameters

stat [Stat](#)

The stat to get the base value of.

Returns

long

The base value of the stat (clamped to the stat's min and max values) if enabled; otherwise, 0

OnLevelDown(EntityLevelChangedContext)

Callback method called when the entity levels down.

```
protected virtual void OnLevelDown(EntityLevelChangedContext context)
```

Parameters

context [EntityLevelChangedContext](#)

The context of the level down event.

OnLevelUp(EntityLevelChangedContext)

Callback method called when the entity levels up.

```
protected virtual void OnLevelUp(EntityLevelChangedContext context)
```

Parameters

context [EntityLevelChangedContext](#)

The context of the level up event.

SetFixed(Stat, long)

Sets the value of a fixed base stat. This is only applicable when [UseClassBaseStats](#) is false.

```
public void SetFixed(Stat stat, long value)
```

Parameters

stat [Stat](#)

The stat to set.

value long

The value to set.

TryGet(Stat, out long)

Tries to get the final value of a stat. Returns true if the stat is contained in the stat set; otherwise, false.

```
public bool TryGet(Stat stat, out long value)
```

Parameters

stat [Stat](#)

The stat to get.

value long

When this method returns, contains the final value of the stat if the stat is contained in the stat set; otherwise, 0.

Returns

bool

true if the stat is contained in the stat set; otherwise, false.

Events

OnStatChangedCallback

Entity-local C# event raised whenever a stat value changes on this component. Fired alongside [OnStat Changed](#) (SO-based) for each stat change. Use this for direct code subscriptions where a ScriptableObject event reference is inconvenient (Currently reserved for future use with **EntityModifiers** stat-driven max stacks trimming).

```
public event Action<StatChangeInfo> OnStatChangedCallback
```

Event Type

Action<[StatChangeInfo](#)>

Interface IHasStatSet

Namespace: [ElectricDrill.AstraRpgFramework.Stats](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that provide a source of stat sets. Defines the contract for accessing a StatSet that contains stat definitions.

```
public interface IHasStatSet
```

Properties

StatSet

Gets the stat set that defines the available stats.

```
StatSet StatSet { get; }
```

Property Value

[StatSet](#)

Interface IStatReader

Namespace: [ElectricDrill.AstraRpgFramework.Stats](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Read-only interface for querying stat values from an entity. Use [TryGet\(Stat, out long\)](#) as the safe access path; [Get](#) requires [Contains\(T\)](#) to be checked first.

```
public interface IStatReader : IValueContainer<Stat>
```

Inherited Members

[IValueContainer<Stat>.Contains\(Stat\)](#)

Methods

TryGet(Stat, out long)

Tries to get the final value of a stat.

```
bool TryGet(Stat stat, out long value)
```

Parameters

stat [Stat](#)

The stat to query.

value long

The final value if the stat is present; otherwise, 0.

Returns

bool

true if the stat is present; otherwise, false.

Class Stat

Namespace: [ElectricDrill.AstraRpgFramework.Stats](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents a stat in the RPG system that extends BoundedValue with attribute scaling capabilities. Stats can be scaled based on entity attributes and provide equality comparison based on name.

```
public class Stat : BoundedValue
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← [BoundedValue](#) ← Stat

Inherited Members

[BoundedValue.HasMaxValue](#) , [BoundedValue.MaxValue](#) , [BoundedValue.HasMinValue](#) , [BoundedValue.MinValue](#)

Properties

AttributesScaling

Gets or sets the attributes scaling component that determines how this stat scales with entity attributes. Can be null if the stat doesn't scale with attributes.

```
[CanBeNull]  
public AttributesScalingComponent AttributesScaling { get; }
```

Property Value

[AttributesScalingComponent](#)

Methods

Equals(object)

```
public override bool Equals(object obj)
```

Parameters

obj object

Returns

bool

GetHashCode()

```
public override int GetHashCode()
```

Returns

int

Operators

operator ==(Stat, Stat)

Determines whether two Stat objects are equal based on their names.

```
public static bool operator ==(Stat a, Stat b)
```

Parameters

a [Stat](#)

The first stat to compare.

b [Stat](#)

The second stat to compare.

Returns

bool

true if the stats are equal; otherwise, false.

operator !=(Stat, Stat)

Determines whether two Stat objects are not equal.

```
public static bool operator !=(Stat a, Stat b)
```

Parameters

a [Stat](#)

The first stat to compare.

b [Stat](#)

The second stat to compare.

Returns

bool

true if the stats are not equal; otherwise, false.

Struct StatChangeInfo

Namespace: [ElectricDrill.AstraRpgFramework.Stats](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Contains information about a stat change event, including the entity, stat, and old/new values. Used for tracking and responding to stat modifications.

```
public struct StatChangeInfo : IHasValueChange<long>
```

Implements

[IHasValueChange](#)<long>

Constructors

StatChangeInfo(EntityStats, Stat, long, long)

Initializes a new instance of the StatChangeInfo structure.

```
public StatChangeInfo(EntityStats entity, Stat stat, long previousValue, long newValue)
```

Parameters

entity [EntityStats](#)

The EntityStats component that owns the stat.

stat [Stat](#)

The stat that was changed.

previousValue long

The previous value of the stat.

newValue long

The new value of the stat.

Fields

EntityStats

The EntityStats component that owns the changed stat.

```
public EntityStats EntityStats
```

Field Value

[EntityStats](#)

Stat

The stat that was changed.

```
public Stat Stat
```

Field Value

[Stat](#)

Properties

AbsAmount

The absolute amount of change between PreviousValue and NewValue.

```
public long AbsAmount { get; }
```

Property Value

long

NewValue

The value of the stat after the change.

```
public readonly long NewValue { get; }
```

Property Value

long

PreviousValue

The value of the stat before the change.

```
public readonly long PreviousValue { get; }
```

Property Value

long

Class StatSet

Namespace: [ElectricDrill.AstraRpgFramework.Stats](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A ScriptableObject that defines a collection of stats that can be used by entities. Implements IStatContainer to provide stat containment functionality.

```
public class StatSet : ScriptableObject, IValueContainer<Stat>, IEditorValidatable
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← StatSet

Implements

[IValueContainer](#)<[Stat](#)>, [IEditorValidatable](#)

Properties

Stats

Gets a read-only list of all stats in this set, including stats from included StatSets. Duplicates are automatically removed.

```
public IReadOnlyList<Stat> Stats { get; }
```

Property Value

IReadOnlyList<[Stat](#)>

Methods

Contains(Stat)

Determines whether this stat set contains the specified stat. Also checks in included StatSets.

```
public virtual bool Contains(Stat stat)
```

Parameters

stat [Stat](#)

The stat to check for.

Returns

bool

true if the stat set contains the stat; otherwise, false.

Contains(StatSet)

Determines whether this stat set contains another stat set. Either directly or indirectly.

```
public virtual bool Contains(StatSet statSet)
```

Parameters

statSet [StatSet](#)

The stat set to check for.

Returns

bool

true if the **statSet** is contained in this stat set; otherwise, false.

Get(Stat)

Gets the stat from this set that matches the specified stat. Also looks in included StatSets.

```
public Stat Get(Stat stat)
```

Parameters

stat [Stat](#)

The stat to find.

Returns

[Stat](#)

The matching stat from this set.

Class StatSetInstance

Namespace: [ElectricDrill.AstraRpgFramework.Stats](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents an instance of a StatSet with actual values for each stat. Implements IEnumerable and IStatContainer to provide collection functionality.

```
public class StatSetInstance : IValueContainer<Stat>
```

Inheritance

object ← StatSetInstance

Implements

[IValueContainer](#) <[Stat](#)>

Constructors

StatSetInstance(StatSet)

Initializes a new StatSetInstance based on the provided StatSet. All stats from the StatSet are initialized with a value of 0.

```
public StatSetInstance(StatSet statSet)
```

Parameters

statSet [StatSet](#)

The StatSet to base this instance on.

Properties

this[Stat]

Gets or sets the value of the specified stat using indexer syntax.

```
public long this[Stat stat] { get; set; }
```

Parameters

stat [Stat](#)

The stat to get or set.

Property Value

long

The current value of the stat.

Methods

AddValue(Stat, long)

Adds value to the specified stat. If the stat doesn't exist, it will be created with the specified value. Use negative values to subtract from the stat.

```
public void AddValue(Stat stat, long value)
```

Parameters

stat [Stat](#)

The stat to modify.

value long

The value to add (can be negative).

Clone()

Creates a deep copy of this StatSetInstance.

```
public StatSetInstance Clone()
```

Returns

[StatSetInstance](#)

A new StatSetInstance with the same stats and values.

Contains(Stat)

Determines whether this instance contains the specified stat.

```
public bool Contains(Stat stat)
```

Parameters

stat [Stat](#)

The stat to check for.

Returns

bool

true if the instance contains the stat; otherwise, false.

Get(Stat)

Gets the current value of the specified stat.

```
public long Get(Stat stat)
```

Parameters

stat [Stat](#)

The stat to retrieve.

Returns

long

The current value of the stat.

GetAsPercentage(Stat)

Gets the value of the specified stat as a Percentage.

```
public Percentage GetAsPercentage(Stat stat)
```

Parameters

stat [Stat](#)

The stat to get as a percentage.

Returns

[Percentage](#)

The stat value wrapped in a Percentage object.

GetEnumerator()

```
public IEnumerable<KeyValuePair<Stat, long>> GetEnumerator()
```

Returns

IEnumerable<KeyValuePair<[Stat](#), long>>

Operators

operator +(StatSetInstance, StatSetInstance)

Adds the values of corresponding stats from two StatSetInstances. All stats present in the first instance must also be present in the second instance.

```
public static StatSetInstance operator +(StatSetInstance a, StatSetInstance b)
```

Parameters

a [StatSetInstance](#)

The first StatSetInstance.

b [StatSetInstance](#)

The second StatSetInstance.

Returns

[StatSetInstance](#)

A new StatSetInstance with the sum of corresponding stat values.

Exceptions

KeyNotFoundException

Thrown when a stat from instance 'a' is not found in instance 'b'.

Class StatToStatModifier

Namespace: [ElectricDrill.AstraRpgFramework.Stats](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A ScriptableObject that defines how one stat can scale upon another stat basing on a percentage. Used to create relationships between different stats in the RPG system.

```
public class StatToStatModifier : ScriptableObject
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← StatToStatModifier

Properties

Percentage

Gets the percentage of the source stat value that will be applied to the target stat.

```
public Percentage Percentage { get; }
```

Property Value

[Percentage](#)

SourceStat

Gets the stat that provides the value for the modification calculation.

```
public Stat SourceStat { get; }
```

Property Value

[Stat](#)

TargetStat

Gets the stat that will be modified by the source stat.

```
public Stat TargetStat { get; }
```

Property Value

[Stat](#)

Namespace ElectricDrill.AstraRpgFramework.

Tags

Classes

[GameTag](#)

Lightweight typed tag identifier. Use as a ScriptableObject reference to tag game entities, modifiers, abilities, or any other game data.

[GameTagSet](#)

Embeddable, serializable collection of [GameTag](#)s. Usable as a `[SerializeField]` in any ScriptableObject, MonoBehaviour, or plain class.

Interfaces

[IActiveModifierTagProvider](#)

Framework-side contract for querying active modifier tags on an entity.

Implemented by `EntityModifiers` (Modifiers package). Placing the interface in Framework allows `TagPresentCondition` (also in Modifiers) to reach it via `GetComponent<IActiveModifierTagProvider>()` without creating a circular dependency (Framework → Modifiers).

[ITaggable](#)

Marks a type as having an associated set of [GameTags](#).

Class GameTag

Namespace: [ElectricDrill.AstraRpgFramework.Tags](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Lightweight typed tag identifier. Use as a ScriptableObject reference to tag game entities, modifiers, abilities, or any other game data.

```
[CreateAssetMenu(fileName = "New Game Tag", menuName = "Astra RPG Framework/Game Tag")]  
public class GameTag : ScriptableObject
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← GameTag

Class GameTagSet

Namespace: [ElectricDrill.AstraRpgFramework.Tags](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Embeddable, serializable collection of [GameTag](#)s. Usable as a `[SerializeField]` in any ScriptableObject, MonoBehaviour, or plain class.

```
[Serializable]
public class GameTagSet
```

Inheritance

object ← GameTagSet

Constructors

GameTagSet()

Creates an empty tag set.

```
public GameTagSet()
```

GameTagSet(GameTag)

Creates a tag set containing a single tag.

```
public GameTagSet(GameTag single)
```

Parameters

single [GameTag](#)

Properties

Tags

Read-only view of the tags in this set.

```
public IReadOnlyList<GameTag> Tags { get; }
```

Property Value

IReadOnlyList<[GameTag](#)>

Methods

Contains(GameTag)

Returns **true** if this set contains **tag**.

```
public bool Contains(GameTag tag)
```

Parameters

tag [GameTag](#)

Returns

bool

ContainsAll(GameTagSet)

Returns **true** if this set contains every tag in **other**.

```
public bool ContainsAll(GameTagSet other)
```

Parameters

other [GameTagSet](#)

Returns

bool

ContainsAny(GameTagSet)

Returns **true** if this set contains at least one tag from **other**.

```
public bool ContainsAny(GameTagSet other)
```

Parameters

other [GameTagSet](#)

Returns

bool

Interface IActiveModifierTagProvider

Namespace: [ElectricDrill.AstraRpgFramework.Tags](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Framework-side contract for querying active modifier tags on an entity.

Implemented by `EntityModifiers` (Modifiers package). Placing the interface in Framework allows `TagPresentCondition` (also in Modifiers) to reach it via `GetComponent<IActiveModifierTagProvider>()` without creating a circular dependency (Framework → Modifiers).

```
public interface IActiveModifierTagProvider
```

Methods

HasActiveModifierMatchingTags(GameTagSet)

Returns `true` if at least one active modifier on this entity has all tags in `tags`.

```
bool HasActiveModifierMatchingTags(GameTagSet tags)
```

Parameters

`tags` [GameTagSet](#)

Returns

`bool`

Interface ITaggable

Namespace: [ElectricDrill.AstraRpgFramework.Tags](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Marks a type as having an associated set of [GameTags](#).

```
public interface ITaggable
```

Properties

Tags

The tags associated with this object.

```
GameTagSet Tags { get; }
```

Property Value

[GameTagSet](#)

Namespace ElectricDrill.AstraRpgFramework.

Triggers

Classes

[ParameterlessGameEventTrigger](#)

[IReactiveTrigger](#) that wraps a parameterless [GameEvent](#) SO. When the event fires, the callback is invoked with `null` as the payload.

This class is [`Serializable`] so it can be embedded as a field in any ScriptableObject or MonoBehaviour. The adapter dictionaries are runtime-only and are initialized lazily (safe after Unity deserialization).

Interfaces

[IReactiveTrigger](#)

Entity-aware reactive event delivery contract. Implementations wrap a game event source and deliver it to per-entity subscribers via a typed `Action<object>` callback. Subscriptions are keyed by entity so each [EntityCore](#) receives only its own notifications.

Interface IReactiveTrigger

Namespace: [ElectricDrill.AstraRpgFramework.Triggers](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Entity-aware reactive event delivery contract. Implementations wrap a game event source and deliver it to per-entity subscribers via a typed `Action<object>` callback. Subscriptions are keyed by entity so each [EntityCore](#) receives only its own notifications.

```
public interface IReactiveTrigger
```

Methods

Subscribe(EntityCore, Action<object>)

Registers `callback` to be invoked when the trigger fires for `entity`. Calling this more than once for the same entity is a no-op.

```
void Subscribe(EntityCore entity, Action<object> callback)
```

Parameters

`entity` [EntityCore](#)

The entity whose context governs when the callback fires.

`callback` `Action<object>`

Invoked with the event payload (may be `null` for parameterless triggers).

Unsubscribe(EntityCore, Action<object>)

Removes the subscription registered for `entity`. Safe to call even if no subscription exists.

```
void Unsubscribe(EntityCore entity, Action<object> callback)
```

Parameters

entity [EntityCore](#)

callback Action<object>

Class ParameterlessGameEventTrigger

Namespace: [ElectricDrill.AstraRpgFramework.Triggers](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

[IReactiveTrigger](#) that wraps a parameterless [GameEvent](#) SO. When the event fires, the callback is invoked with `null` as the payload.

This class is `[Serializable]` so it can be embedded as a field in any ScriptableObject or MonoBehaviour. The adapter dictionaries are runtime-only and are initialized lazily (safe after Unity deserialization).

```
[Serializable]
public class ParameterlessGameEventTrigger : IReactiveTrigger
```

Inheritance

object ← ParameterlessGameEventTrigger

Implements

[IReactiveTrigger](#)

Methods

Subscribe(EntityCore, Action<object>)

Registers `callback` to be invoked when the trigger fires for `entity`. Calling this more than once for the same entity is a no-op.

```
public void Subscribe(EntityCore entity, Action<object> callback)
```

Parameters

`entity` [EntityCore](#)

The entity whose context governs when the callback fires.

`callback` Action<object>

Invoked with the event payload (may be `null` for parameterless triggers).

Unsubscribe(EntityCore, Action<object>)

Removes the subscription registered for `entity`. Safe to call even if no subscription exists.

```
public void Unsubscribe(EntityCore entity, Action<object> callback)
```

Parameters

`entity` [EntityCore](#)

`callback` Action<object>

Namespace ElectricDrill.AstraRpgFramework.

Utils

Classes

[BoundedValue](#)

Abstract base class for values that can be bounded by minimum and maximum limits. Provides functionality to define optional min/max constraints and clamp values within those bounds.

[Cache<KType, VType>](#)

A generic cache class that provides basic caching functionality.

[GrowthFormula](#)

Represents a formula to calculate growth values for different levels, up to a maximum level.

[InitializationUtils](#)

Provides utility methods for initializing and refreshing collections during object setup. Contains helper methods commonly used during component initialization and validation.

[IntRef](#)

A reference to an integer value. Can either be a constant value or a reference to an [IntVar](#) ScriptableObject.

[IntVar](#)

ScriptableObject that holds an integer value. Can be used to share an integer value between different GameObjects and scenes.

[LongRef](#)

A reference to a long value. Can either be a constant value or a reference to a [LongVar](#) ScriptableObject.

[LongVar](#)

ScriptableObject that holds a long value. Can be used to share a long value between different GameObjects and scenes.

[Percentage](#)

The Percentage class represents a percentage value and provides various operators and conversions. Implicit long to Percentage value conversion is available. To express a 100% value, use 100L. Implicit Percentage to double conversion is available. When doing so, the percentage is automatically divided by 100.

[SerializableDictionary<TKey, TValue>](#)

A serializable dictionary implementation that can be displayed and edited in the Unity Inspector. Provides all standard dictionary functionality while maintaining Unity serialization compatibility.

[SerializableHashSet<T>](#)

A serializable hash set implementation that can be displayed and edited in the Unity Inspector. Provides all standard hash set functionality while maintaining Unity serialization compatibility.

[TypeSelectableAttribute](#)

Custom attribute to mark fields for SerializeReference type selection. Apply this to [SerializeReference] fields to enable type selection dropdown.

[ValidationError](#)

Represents a validation error that occurred during formula evaluation.

Structs

[SerKeyValPair<T, U>](#)

A serializable key-value pair structure that can be used in Unity's inspector. Provides implicit conversion to and from the standard KeyValuePair.

Interfaces

[IEditorValidatable](#)

Interface for objects that support validation events in the Unity Editor. This allows other components to subscribe and react when an object is validated (typically through OnValidate in the inspector).

[IValueContainer<T>](#)

Interface for objects that can contain and query values. Provides a standardized way to check for value containment across different value containers.

[IValueProvider<KeyType, ReturnType>](#)

Provides a method to retrieve a value based on a key.

Class BoundedValue

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Abstract base class for values that can be bounded by minimum and maximum limits. Provides functionality to define optional min/max constraints and clamp values within those bounds.

```
public abstract class BoundedValue : ScriptableObject
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← BoundedValue

Derived

[Attribute](#), [Stat](#)

Properties

HasMaxValue

Gets or sets whether this value has a maximum limit.

```
public bool HasMaxValue { get; }
```

Property Value

bool

HasMinValue

Gets or sets whether this value has a minimum limit.

```
public bool HasMinValue { get; }
```

Property Value

bool

MaxValue

Gets or sets the maximum allowed value when HasMaxValue is true.

```
public long MaxValue { get; }
```

Property Value

long

MinValue

Gets or sets the minimum allowed value when HasMinValue is true.

```
public int MinValue { get; }
```

Property Value

int

Class Cache<KType, VType>

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A generic cache class that provides basic caching functionality.

```
public class Cache<KType, VType>
```

Type Parameters

KType

The type of the keys in the cache.

VType

The type of the values in the cache.

Inheritance

```
object ← Cache<KType, VType>
```

Properties

this[KType]

Gets or sets the value for the specified key in the cache.

```
public VType this[KType key] { get; set; }
```

Parameters

key KType

The key of the item to get or set.

Property Value

VType

The value associated with the specified key.

Methods

Get(KType)

Gets the value for the specified key from the cache. If the key is not found, a `KeyNotFoundException` is thrown.

```
public VType Get(KType key)
```

Parameters

key KType

The key of the item to get.

Returns

VType

The value associated with the specified key.

Exceptions

`KeyNotFoundException`

Thrown when the key is not found in the cache.

Has(KType)

Determines whether the cache contains an item with the specified key.

```
public bool Has(KType key)
```

Parameters

key KType

The key to locate in the cache.

Returns

bool

true if the cache contains an item with the key; otherwise, false.

Invalidate(KType)

Invalidates the cache item for the specified key. If key is not found, nothing happens.

```
public void Invalidate(KType key)
```

Parameters

key KType

The key of the item to invalidate.

InvalidateAll()

Invalidates all items in the cache.

```
public void InvalidateAll()
```

Set(KType, VType)

Sets the value for the specified key in the cache. UPSERT operation.

```
public void Set(KType key, VType value)
```

Parameters

key KType

The key of the item to set.

value VType

The value to set for the specified key.

TryGet(KType, out VType)

Tries to get the value for the specified key from the cache.

```
public bool TryGet(KType key, out VType value)
```

Parameters

key KType

The key of the item to get.

value VType

When this method returns, contains the value associated with the specified key, if the key is found; otherwise, the default value for the type of the value parameter.

Returns

bool

true if the cache contains an item with the specified key; otherwise, false.

Class GrowthFormula

Namespace: [ElectricDrill.AstraRpgFramework.Utls](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents a formula to calculate growth values for different levels, up to a maximum level.

```
public class GrowthFormula : ScriptableObject
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← GrowthFormula

Properties

GrowthFoValues

Gets the pre-calculated growth formula values for each level. Values are refreshed on validation. The array is indexed from 0 to maxLevel - 1, where index 0 corresponds to level 1.

```
public double[] GrowthFoValues { get; }
```

Property Value

double[]

ValidationErrors

Gets the validation errors that occurred during the last formula evaluation.

```
public List<ValidationError> ValidationErrors { get; }
```

Property Value

List<[ValidationError](#)>

Methods

GetGrowthValue(int)

Gets the growth value for a specific level. Levels start from 1.

```
public long GetGrowthValue(int level)
```

Parameters

level int

The level for which to get the growth value.

Returns

long

The growth value for the specified level.

Interface IEditorValidatable

Namespace: [ElectricDrill.AstraRpgFramework.Utls](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that support validation events in the Unity Editor. This allows other components to subscribe and react when an object is validated (typically through OnValidate in the inspector).

```
public interface IEditorValidatable
```

Interface IValueContainer<T>

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Interface for objects that can contain and query values. Provides a standardized way to check for value containment across different value containers.

```
public interface IValueContainer<in T>
```

Type Parameters

T

The type of value contained and queried by the container.

Methods

Contains(T)

Determines whether the container includes the specified value.

```
bool Contains(T value)
```

Parameters

value T

The value to locate in the container.

Returns

bool

True if the value is found in the container; otherwise, false.

Interface IValueProvider<KeyType, ReturnType>

Namespace: [ElectricDrill.AstraRpgFramework.Utls](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Provides a method to retrieve a value based on a key.

```
public interface IValueProvider<in KeyType, out ReturnType>
```

Type Parameters

KeyType

The type of key used to retrieve the value.

ReturnType

The type of value returned by the provider.

Methods

Get(KeyType)

Gets the value associated with the specified key.

```
ReturnType Get(KeyType key)
```

Parameters

key KeyType

The key used to retrieve the value.

Returns

ReturnType

The value associated with the specified key.

Class InitializationUtils

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Provides utility methods for initializing and refreshing collections during object setup. Contains helper methods commonly used during component initialization and validation.

```
public static class InitializationUtils
```

Inheritance

object ← InitializationUtils

Methods

RefreshInspectorReservedValues<TKey, TValue>(ref List<SerKeyValPair<TKey, TValue>>, IEnumerable<TKey>)

Refreshes a list of key-value pairs to match a provided set of keys, preserving existing values where possible. This method ensures that the inspector-reserved values list contains exactly the keys specified, maintaining any previously assigned values and setting defaults for new keys.

```
public static void RefreshInspectorReservedValues<TKey, TValue>(ref List<SerKeyValPair<TKey, TValue>> inspectorReservedValues, IEnumerable<TKey> keys)
```

Parameters

inspectorReservedValues List<[SerKeyValPair](#)<TKey, TValue>>

The list to refresh (passed by reference)

keys IEnumerable<TKey>

The collection of keys that should be present in the list

Type Parameters

TKey

The type of the keys

TValue

The type of the values

Class IntRef

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A reference to an integer value. Can either be a constant value or a reference to an [IntVar](#) ScriptableObject.

```
[Serializable]
public class IntRef
```

Inheritance

object ← IntRef

Fields

ConstantValue

The constant integer value.

```
public int ConstantValue
```

Field Value

int

UseConstant

If true, uses the constant value. Otherwise, uses the [IntVar](#) variable.

```
public bool UseConstant
```

Field Value

bool

Variable

The [IntVar](#) variable.

```
public IntVar Variable
```

Field Value

[IntVar](#)

Properties

Value

Gets or sets the value of the reference.

```
public int Value { get; set; }
```

Property Value

int

Operators

implicit operator int(IntRef)

Allows implicit conversion from an IntRef to an int.

```
public static implicit operator int(IntRef reference)
```

Parameters

reference [IntRef](#)

The IntRef to convert.

Returns

int

implicit operator IntRef(int)

Allows implicit conversion from an int to an IntRef, creating a constant reference.

```
public static implicit operator IntRef(int value)
```

Parameters

value int

The integer value.

Returns

[IntRef](#)

Class IntVar

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

ScriptableObject that holds an integer value. Can be used to share an integer value between different GameObjects and scenes.

```
[CreateAssetMenu(fileName = "Int Var", menuName = "Astra RPG Framework/Utils/Int Var")]  
public class IntVar : ScriptableObject
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← IntVar

Properties

Value

The integer value.

```
public int Value { get; set; }
```

Property Value

int

Operators

implicit operator int(IntVar)

Allows implicit conversion from an IntVar to an int.

```
public static implicit operator int(IntVar var)
```

Parameters

var [IntVar](#)

The IntVar to convert.

Returns

int

Class LongRef

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A reference to a long value. Can either be a constant value or a reference to a [LongVar](#) ScriptableObject.

```
[Serializable]  
public class LongRef
```

Inheritance

object ← LongRef

Fields

ConstantValue

The constant long value.

```
public long ConstantValue
```

Field Value

long

UseConstant

If true, uses the constant value. Otherwise, uses the [LongVar](#) variable.

```
public bool UseConstant
```

Field Value

bool

Variable

The [LongVar](#) variable.

```
public LongVar Variable
```

Field Value

[LongVar](#)

Properties

Value

Gets or sets the value of the reference.

```
public long Value { get; set; }
```

Property Value

long

Operators

implicit operator long(LongRef)

Allows implicit conversion from a LongRef to a long.

```
public static implicit operator long(LongRef reference)
```

Parameters

reference [LongRef](#)

The LongRef to convert.

Returns

long

implicit operator LongRef(int)

Allows implicit conversion from an int to an LongRef, creating a constant reference.

```
public static implicit operator LongRef(int value)
```

Parameters

value int

The int value.

Returns

[LongRef](#)

implicit operator LongRef(long)

Allows implicit conversion from a long to an LongRef, creating a constant reference.

```
public static implicit operator LongRef(long value)
```

Parameters

value long

The long value.

Returns

[LongRef](#)

Class LongVar

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

ScriptableObject that holds a long value. Can be used to share a long value between different GameObjects and scenes.

```
[CreateAssetMenu(fileName = "Long Var", menuName = "Astra RPG Framework/Utils/Long Var")]  
public class LongVar : ScriptableObject
```

Inheritance

object ← [Object](#) ← [ScriptableObject](#) ← LongVar

Properties

Value

The long value.

```
public long Value { get; set; }
```

Property Value

long

Operators

implicit operator long(LongVar)

Allows implicit conversion from a LongVar to a long.

```
public static implicit operator long(LongVar var)
```

Parameters

var [LongVar](#)

The LongVar to convert.

Returns

long

Class Percentage

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

The Percentage class represents a percentage value and provides various operators and conversions. Implicit long to Percentage value conversion is available. To express a 100% value, use 100L. Implicit Percentage to double conversion is available. When doing so, the percentage is automatically divided by 100.

```
[Serializable]  
public class Percentage
```

Inheritance

object ← Percentage

Constructors

Percentage(long)

Initializes a new instance of the Percentage class with the specified value. To express a 100% value, use 100L.

```
public Percentage(long value)
```

Parameters

value long

The value of the percentage.

Methods

CompareTo(Percentage)

Compares the current Percentage instance with another Percentage instance.

```
public int CompareTo(Percentage other)
```

Parameters

other [Percentage](#)

The other percentage to compare to.

Returns

int

An integer indicating the relative order of the percentages.

Equals(object)

```
public override bool Equals(object obj)
```

Parameters

obj object

Returns

bool

GetHashCode()

```
public override int GetHashCode()
```

Returns

int

ToString()

Returns a string representation of the percentage value.

```
public override string ToString()
```

Returns

string

A string representing the percentage value.

Operators

operator +(Percentage, Percentage)

Overrides the + operator to add two Percentage instances.

```
public static Percentage operator +(Percentage a, Percentage b)
```

Parameters

a [Percentage](#)

The first percentage.

b [Percentage](#)

The second percentage.

Returns

[Percentage](#)

A new Percentage instance representing the sum.

operator ==(Percentage, int)

```
public static bool operator ==(Percentage a, int b)
```

Parameters

a [Percentage](#)

b int

Returns

bool

operator ==(Percentage, long)

```
public static bool operator ==(Percentage a, long b)
```

Parameters

a [Percentage](#)

b long

Returns

bool

explicit operator long(Percentage)

Explicit conversion from Percentage to long. The conversion does not divide the value by 100.

```
public static explicit operator long(Percentage percentage)
```

Parameters

percentage [Percentage](#)

The percentage to convert.

Returns

long

operator >(Percentage, int)

```
public static bool operator >(Percentage a, int b)
```

Parameters

a [Percentage](#)

b int

Returns

bool

operator >(Percentage, long)

```
public static bool operator >(Percentage a, long b)
```

Parameters

a [Percentage](#)

b long

Returns

bool

operator >=(Percentage, int)

```
public static bool operator >=(Percentage a, int b)
```

Parameters

a [Percentage](#)

b int

Returns

bool

operator >=(Percentage, long)

```
public static bool operator >=(Percentage a, long b)
```

Parameters

a [Percentage](#)

b long

Returns

bool

implicit operator double(Percentage)

Implicit conversion from Percentage to double. The conversion automatically divides the value by 100.

```
public static implicit operator double(Percentage percentage)
```

Parameters

percentage [Percentage](#)

The percentage to convert.

Returns

double

implicit operator Percentage(long)

Implicit conversion from long to Percentage. To express a 100% value, use 100L.

```
public static implicit operator Percentage(long value)
```

Parameters

value long

The value to convert.

Returns

[Percentage](#)

operator !=(Percentage, int)

```
public static bool operator !=(Percentage a, int b)
```

Parameters

a [Percentage](#)

b int

Returns

bool

operator !=(Percentage, long)

```
public static bool operator !=(Percentage a, long b)
```

Parameters

a [Percentage](#)

b long

Returns

bool

operator <(Percentage, int)

```
public static bool operator <(Percentage a, int b)
```

Parameters

a [Percentage](#)

b int

Returns

bool

operator <(Percentage, long)

```
public static bool operator <(Percentage a, long b)
```

Parameters

a [Percentage](#)

b long

Returns

bool

operator <=(Percentage, int)

```
public static bool operator <=(Percentage a, int b)
```

Parameters

a [Percentage](#)

b int

Returns

bool

operator <=(Percentage, long)

```
public static bool operator <=(Percentage a, long b)
```

Parameters

a [Percentage](#)

b long

Returns

bool

operator -(Percentage, Percentage)

Overrides the - operator to subtract one Percentage from another.

```
public static Percentage operator -(Percentage a, Percentage b)
```

Parameters

a [Percentage](#)

The first percentage.

b [Percentage](#)

The second percentage.

Returns

[Percentage](#)

A new Percentage instance representing the difference.

operator -(Percentage)

Overrides the unary - operator to negate a Percentage.

```
public static Percentage operator -(Percentage a)
```

Parameters

a [Percentage](#)

The percentage to negate.

Returns

[Percentage](#)

A new Percentage instance representing the negated value.

Struct SerKeyValPair<T, U>

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A serializable key-value pair structure that can be used in Unity's inspector. Provides implicit conversion to and from the standard KeyValuePair.

```
[Serializable]  
public struct SerKeyValPair<T, U>
```

Type Parameters

T

The type of the key.

U

The type of the value.

Constructors

SerKeyValPair(T, U)

Initializes a new instance of the SerKeyValPair structure with the specified key and value.

```
public SerKeyValPair(T key, U value)
```

Parameters

key T

The key of the key-value pair.

value U

The value of the key-value pair.

Fields

Key

The key of the key-value pair.

```
public T Key
```

Field Value

T

Value

The value of the key-value pair.

```
public U Value
```

Field Value

U

Operators

implicit operator KeyValuePair<T, U>(SerKeyValPair<T, U>)

Implicitly converts a SerKeyValPair to a KeyValuePair.

```
public static implicit operator KeyValuePair<T, U>(SerKeyValPair<T, U> serKeyValPair)
```

Parameters

serKeyValPair [SerKeyValPair](#)<T, U>

The SerKeyValPair to convert.

Returns

KeyValuePair<T, U>

A KeyValuePair with the same key and value.

implicit operator SerKeyValPair<T, U>(KeyValuePair<T, U>)

Implicitly converts a KeyValuePair to a SerKeyValPair.

```
public static implicit operator SerKeyValPair<T, U>(KeyValuePair<T, U> keyValuePair)
```

Parameters

keyValuePair KeyValuePair<T, U>

The KeyValuePair to convert.

Returns

[SerKeyValPair](#)<T, U>

A SerKeyValPair with the same key and value.

Class SerializableDictionary<TKey, TValue>

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A serializable dictionary implementation that can be displayed and edited in the Unity Inspector. Provides all standard dictionary functionality while maintaining Unity serialization compatibility.

```
[Serializable]  
public class SerializableDictionary<TKey, TValue>
```

Type Parameters

TKey

The type of the dictionary keys

TValue

The type of the dictionary values

Inheritance

object ← SerializableDictionary<TKey, TValue>

Properties

this[TKey]

Gets or sets the value associated with the specified key.

```
public TValue this[TKey key] { get; set; }
```

Parameters

key TKey

The key of the value to get or set

Property Value

TValue

The value associated with the specified key

Keys

Gets a collection containing the keys in the dictionary.

```
public Dictionary<TKey, TValue>.KeyCollection Keys { get; }
```

Property Value

Dictionary<TKey, TValue>.KeyCollection

Values

Gets a collection containing the values in the dictionary.

```
public Dictionary<TKey, TValue>.ValueCollection Values { get; }
```

Property Value

Dictionary<TKey, TValue>.ValueCollection

Methods

Clear()

Removes all keys and values from the dictionary.

```
public void Clear()
```

ContainsKey(TKey)

Determines whether the dictionary contains the specified key.

```
public bool ContainsKey(TKey key)
```

Parameters

key TKey

The key to locate

Returns

bool

True if the dictionary contains the key; otherwise, false

GetEnumerator()

Returns an enumerator that iterates through the dictionary.

```
public IEnumerator<KeyValuePair<TKey, TValue>> GetEnumerator()
```

Returns

IEnumerator<KeyValuePair<TKey, TValue>>

An enumerator for the dictionary

OnAfterDeserialize()

Reconstructs the internal dictionary from the serialized list format. Called automatically by Unity after deserialization occurs.

```
public void OnAfterDeserialize()
```

OnBeforeSerialize()

Converts the internal dictionary to a serialized list format for Unity serialization. Called automatically by Unity before serialization occurs.

```
public void OnBeforeSerialize()
```

TryGetValue(TKey, out TValue)

Attempts to get the value associated with the specified key.

```
public bool TryGetValue(TKey key, out TValue value)
```

Parameters

key TKey

The key to locate

value TValue

The value associated with the key, if found

Returns

bool

True if the key was found; otherwise, false

Operators

implicit operator Dictionary<TKey, TValue>
(SerializableDictionary<TKey, TValue>)

```
public static implicit operator Dictionary<TKey, TValue>(SerializableDictionary<TKey,  
TValue> serializableDictionary)
```

Parameters

serializableDictionary [SerializableDictionary](#)<TKey, TValue>

Returns

Dictionary<TKey, TValue>

implicit operator SerializableDictionary<TKey, TValue>
(Dictionary<TKey, TValue>)

```
public static implicit operator SerializableDictionary<TKey, TValue>(Dictionary<TKey,  
TValue> dictionary)
```

Parameters

dictionary Dictionary<TKey, TValue>

Returns

[SerializableDictionary](#)<TKey, TValue>

Class SerializableHashSet<T>

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

A serializable hash set implementation that can be displayed and edited in the Unity Inspector. Provides all standard hash set functionality while maintaining Unity serialization compatibility.

```
[Serializable]  
public class SerializableHashSet<T>
```

Type Parameters

T

The type of elements in the hash set

Inheritance

object $\leftarrow$ SerializableHashSet<T>

Properties

Count

Gets the number of elements contained in the hash set.

```
public int Count { get; }
```

Property Value

int

IsReadOnly

Gets a value indicating whether the hash set is read-only.

```
public bool IsReadOnly { get; }
```

Property Value

bool

Methods

Add(T)

Adds the specified element to the hash set.

```
public void Add(T item)
```

Parameters

item T

The element to add

Clear()

Removes all elements from the hash set.

```
public void Clear()
```

Contains(T)

Determines whether the hash set contains the specified element.

```
public bool Contains(T item)
```

Parameters

item T

The element to locate

Returns

bool

True if the hash set contains the element; otherwise, false

CopyTo(T[], int)

Copies the elements of the hash set to an array, starting at the specified array index.

```
public void CopyTo(T[] array, int arrayIndex)
```

Parameters

array T[]

The destination array

arrayIndex int

The zero-based index in array at which copying begins

GetEnumerator()

Returns an enumerator that iterates through the hash set.

```
public IEnumerator<T> GetEnumerator()
```

Returns

IEnumerator<T>

An enumerator for the hash set

GetObjectData(SerializationInfo, StreamingContext)

Populates a SerializationInfo with the data needed to serialize the hash set.

```
public void GetObjectData(SerializationInfo info, StreamingContext context)
```

Parameters

info SerializationInfo

The SerializationInfo to populate

context StreamingContext

The destination for this serialization

OnAfterDeserialize()

Reconstructs the internal hash set from the serialized list format. Called automatically by Unity after deserialization occurs.

```
public void OnAfterDeserialize()
```

OnBeforeSerialize()

Converts the internal hash set to a serialized list format for Unity serialization. Called automatically by Unity before serialization occurs.

```
public void OnBeforeSerialize()
```

Remove(T)

Removes the specified element from the hash set.

```
public bool Remove(T item)
```

Parameters

item T

The element to remove

Returns

bool

True if the element was successfully removed; otherwise, false

RemoveWhere(Predicate<T>)

Removes all elements that match the conditions defined by the specified predicate.

```
public int RemoveWhere(Predicate<T> match)
```

Parameters

match Predicate<T>

The predicate that defines the conditions of the elements to remove

Returns

int

The number of elements that were removed

Class TypeSelectableAttribute

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Custom attribute to mark fields for SerializeReference type selection. Apply this to [SerializeReference] fields to enable type selection dropdown.

```
public class TypeSelectableAttribute : PropertyAttribute
```

Inheritance

object ← Attribute ← [PropertyAttribute](#) ↗ ← TypeSelectableAttribute

Constructors

TypeSelectableAttribute()

Creates a TypeSelectable attribute without a default type.

```
public TypeSelectableAttribute()
```

TypeSelectableAttribute(Type)

Creates a TypeSelectable attribute with a specified default type.

```
public TypeSelectableAttribute(Type defaultType)
```

Parameters

defaultType Type

The type to instantiate when "Default" is selected.

Properties

DefaultType

The default type to use when "Default" is selected. If null, "None" option will be shown instead.

```
public Type DefaultType { get; }
```

Property Value

Type

Class ValidationError

Namespace: [ElectricDrill.AstraRpgFramework.Utils](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Represents a validation error that occurred during formula evaluation.

[Serializable]

```
public class ValidationError
```

Inheritance

object ← ValidationError

Fields

AffectedLevels

```
public List<int> AffectedLevels
```

Field Value

List<int>

DetailedMessage

```
public string DetailedMessage
```

Field Value

string

ErrorType

```
public string ErrorType
```

Field Value

string

FormulaExpression

```
public string FormulaExpression
```

Field Value

string

Namespace ElectricDrill.AstraRpgFramework. Utils.UI

Classes

[DefaultAttributeCompactDisplaySOnameProvider](#)

Attribute display name provider that returns a compact version of the attribute's name.

[DefaultDisplaySOnameProvider<T>](#)

Default implementation of IDisplayNameProvider that returns the asset's name property.

Interfaces

[IDisplaySOnameProvider<T>](#)

Provides display names for ScriptableObject assets. This interface allows for flexible presentation strategies, including localization. Add [Serializable] to implementations to suppress Unity warnings.

Class

DefaultAttributeCompactDisplaySOnameProvider

Namespace: [ElectricDrill.AstraRpgFramework.Utls.UI](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Attribute display name provider that returns a compact version of the attribute's name.

```
[Serializable]
public class DefaultAttributeCompactDisplaySOnameProvider :
    IDisplaySOnameProvider<Attribute>
```

Inheritance

object ← DefaultAttributeCompactDisplaySOnameProvider

Implements

[IDisplaySOnameProvider](#) <[Attribute](#)>

Methods

GetDisplayName(Attribute)

Gets the display name by returning the first three characters of the attribute's name in uppercase.

```
public string GetDisplayName(Attribute attribute)
```

Parameters

attribute [Attribute](#)

The attribute to get the compact display name for.

Returns

string

The compact display name.

Class DefaultDisplayNameProvider<T>

Namespace: [ElectricDrill.AstraRpgFramework.Utils.UI](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Default implementation of IDisplayNameProvider that returns the asset's name property.

```
[Serializable]  
public class DefaultDisplayNameProvider<T> : IDisplayDisplayNameProvider<T> where T  
: ScriptableObject
```

Type Parameters

T

The type of ScriptableObject.

Inheritance

object ← DefaultDisplayNameProvider<T>

Implements

[IDisplayDisplayNameProvider](#)<T>

Methods

GetDisplayName(T)

Gets the display name using the asset's name, or "None" if null.

```
public string GetDisplayName(T asset)
```

Parameters

asset T

The asset to get the display name for.

Returns

string

The asset's name or "None" if the asset is null.

Interface IDisplaySOnNameProvider<T>

Namespace: [ElectricDrill.AstraRpgFramework.Utls.UI](#)

Assembly: com.electricdrill.astra-rpg-framework.Runtime.dll

Provides display names for ScriptableObject assets. This interface allows for flexible presentation strategies, including localization. Add [Serializable] to implementations to suppress Unity warnings.

```
public interface IDisplaySOnNameProvider<in T> where T : ScriptableObject
```

Type Parameters

T

The type of ScriptableObject to provide display names for.

Methods

GetDisplayName(T)

Gets the display name for the specified asset.

```
string GetDisplayName(T asset)
```

Parameters

asset T

The asset to get the display name for.

Returns

string

The display name for the asset.